STUDENT LABORATORY MANUAL

Module 7: Introduction to Computer Graphics and Visual Computing

20 Guided Examples • 20 Practice Exercises • 20 Outputs

Course: CS0045 – Computer Graphics and Visual Computing

Companion: Module 7 Student Guide

Toolchain: VS Code + MSYS2 UCRT64 GCC + GLEW + FreeGLUT

Student Name:	**Ebron, Nicole Anne V.**
Section:	**TN35**
Professor:	**Dr. Hadji J. Tejuco**

GitHub Repository: https://github.com/nicoleanneebron-art/Ebron-Nicole-Anne-V.

How to Use This Manual

This manual has three parts:

- Part A — 20 worked Examples (Ex01–Ex20). Each is a complete, compiled, ready-to-run program demonstrating one concept at a time: glTranslate/glRotate/glScale, glPushMatrix/glPopMatrix, transformation order, glViewport, projection types (orthographic and perspective), gluLookAt, and the full lecture capstone combining a color VBO pyramid with animation. Type or copy each one into your SOURCE CODE\M7 folder, compile it (Ctrl+Shift+B), and run it (F5) before moving to the next.
- Part B — 20 Practice Exercises (Q01–Q20), split into Easy (Q01–Q07), Medium (Q08–Q14), and Hard (Q15–Q20). Each exercise gives you an Objective, a Task question describing exactly what your program must do, and Hints to point you in the right direction. Read the task and hints, then write the program YOURSELF before checking Part C.
- Part C — the full Answer Key for every exercise in Part B, with complete, compiled, working code.

Setup reminder

This module builds on Modules 5 and 6's GLEW + FreeGLUT setup. On Windows/MSYS2 UCRT64: pacman -S mingw-w64-ucrt-x86_64-glew. Include <GL/glew.h> BEFORE <GL/glut.h>, and call glewInit() after glutCreateWindow()/glutDisplayFunc() but before glutMainLoop(). All programs in this manual use the same cross-platform header block shown below.

```
#define GL_SILENCE_DEPRECATION
#ifdef __APPLE_
#include <GLUT/glut.h>
#else
#include <GL/glew.h>
#include <GL/glut.h>
#endif
#include  <iostream>
using namespace std;
```

Every program in this manual has been compiled and verified with g++ against GLEW and FreeGLUT.

Part A — Guided Examples (Ex01–Ex20)

Example 01 — Baseline Triangle (No Transformations)

Concept: A plain triangle with no transformation calls -- the reference shape every later example moves, rotates, or scales.

Expected output: A solid green triangle, centered on screen.

```
#define GL_SILENCE_DEPRECATION
#ifdef __APPLE_
#include <GLUT/glut.h>
#else
#include <GL/glew.h>
#include <GL/glut.h>
#endif
#include <iostream>
#include <cmath>
#include <cstdlib>
using namespace std;

#define BUFFER_OFFSET(i) ((void*)(i))


// A plain triangle, no transformations applied -- the reference shape
// every later example in this module will move, rotate, or scale.
void display(){
    glClear(GL_COLOR_BUFFER_BIT);
    glColor3f(0.2f, 0.8f, 0.3f);
    glBegin(GL_TRIANGLES);
        glVertex2f( 0.0f,  0.5f);
        glVertex2f(-0.5f, -0.5f);
        glVertex2f( 0.5f, -0.5f);
    glEnd();
    glFlush();
}

int main(int argc, char** argv){
    glutInit(&argc, argv);
    glutInitWindowSize(800, 600);
    glutInitDisplayMode(GLUT_RGBA);
    glutCreateWindow("simple");
    glutDisplayFunc(display);

#ifndef __APPLE_
    GLenum err = glewInit();
    if(err == GLEW_OK){
        glutMainLoop();
    }
#else
```

```
    glutMainLoop();
#endif
}
```

Example 02 — glTranslatef(): Moving an Object

Concept: glMatrixMode(GL_MODELVIEW) + glLoadIdentity() + glTranslatef() shift a triangle away from its raw coordinates.

Expected output: An orange triangle, shifted 0.4 units to the right of center.

```
#define GL_SILENCE_DEPRECATION
#ifdef __APPLE_
#include <GLUT/glut.h>
#else
#include <GL/glew.h>
#include <GL/glut.h>
#endif
#include <iostream>
#include <cmath>
#include <cstdlib>
using namespace std;

#define BUFFER_OFFSET(i) ((void*)(i))


// glTranslate() moves (translates) an object along the x, y, and z axes.
// Here it shifts the triangle 0.4 units to the right of its raw coordinates.
void display(){
    glClear(GL_COLOR_BUFFER_BIT);

    glMatrixMode(GL_MODELVIEW);
    glLoadIdentity();
    glTranslatef(0.4f, 0.0f, 0.0f);

    glColor3f(0.9f, 0.4f, 0.1f);
    glBegin(GL_TRIANGLES);
        glVertex2f( 0.0f,  0.3f);
        glVertex2f(-0.3f, -0.3f);
        glVertex2f( 0.3f, -0.3f);
    glEnd();
    glFlush();
}

int main(int argc, char** argv){
    glutInit(&argc, argv);
    glutInitWindowSize(800, 600);
    glutInitDisplayMode(GLUT_RGBA);
    glutCreateWindow("simple");
```

```
    glutDisplayFunc(display);

#ifndef __APPLE_
    GLenum err = glewInit();
    if(err == GLEW_OK){
        glutMainLoop();
    }
#else
    glutMainLoop();
#endif
}
```

Example 03 — glRotatef(): Rotating an Object

Concept: glRotatef(angle, x, y, z) rotates a shape around a specified axis -- here, 45 degrees around Z.

Expected output: A blue triangle, visibly rotated 45 degrees from its normal upright orientation.

```
#define GL_SILENCE_DEPRECATION
#ifdef __APPLE_
#include <GLUT/glut.h>
#else
#include <GL/glew.h>
#include <GL/glut.h>
#endif
#include <iostream>
#include <cmath>
#include <cstdlib>
using namespace std;

#define BUFFER_OFFSET(i) ((void*)(i))



// glRotate(angle, x, y, z) rotates an object around a specified axis.
// This rotates the triangle 45 degrees around the Z axis, a fixed amount.
void display(){
    glClear(GL_COLOR_BUFFER_BIT);

    glMatrixMode(GL_MODELVIEW);
    glLoadIdentity();
    glRotatef(45.0f, 0.0f, 0.0f, 1.0f);

    glColor3f(0.2f, 0.5f, 0.9f);
    glBegin(GL_TRIANGLES);
        glVertex2f( 0.0f,  0.5f);
        glVertex2f(-0.5f, -0.5f);
        glVertex2f( 0.5f, -0.5f);
    glEnd();
    glFlush();
```

```
}

int main(int argc, char** argv){
    glutInit(&argc, argv);
    glutInitWindowSize(800, 600);
    glutInitDisplayMode(GLUT_RGBA);
    glutCreateWindow("simple");
    glutDisplayFunc(display);

#ifndef __APPLE_
    GLenum err = glewInit();
    if(err == GLEW_OK){
        glutMainLoop();
    }
#else
    glutMainLoop();
#endif
}
```

Example 04 — glScalef(): Resizing an Object

Concept: glScalef(x, y, z) scales a shape along each axis independently -- here, doubling the X width only.

Expected output: A pink/purple triangle, twice as wide as its raw coordinates would normally produce.

```
#define GL_SILENCE_DEPRECATION
#ifdef __APPLE_
#include <GLUT/glut.h>
#else
#include <GL/glew.h>
#include <GL/glut.h>
#endif
#include <iostream>
#include <cmath>
#include <cstdlib>
using namespace std;

#define BUFFER_OFFSET(i) ((void*)(i))


// glScale(x, y, z) scales an object along the x, y, and z axes.
// This doubles the width of the triangle.
void display(){
    glClear(GL_COLOR_BUFFER_BIT);

    glMatrixMode(GL_MODELVIEW);
    glLoadIdentity();
    glScalef(2.0f, 1.0f, 1.0f);
```

```
    glColor3f(0.8f, 0.2f, 0.6f);
    glBegin(GL_TRIANGLES);
        glVertex2f( 0.0f,  0.3f);
        glVertex2f(-0.3f, -0.3f);
        glVertex2f( 0.3f, -0.3f);
    glEnd();
    glFlush();
}

int main(int argc, char** argv){
    glutInit(&argc, argv);
    glutInitWindowSize(800, 600);
    glutInitDisplayMode(GLUT_RGBA);
    glutCreateWindow("simple");
    glutDisplayFunc(display);

#ifndef __APPLE_
    GLenum err = glewInit();
    if(err == GLEW_OK){
        glutMainLoop();
    }
#else
    glutMainLoop();
#endif
}
```

Example 05 — glPushMatrix() / glPopMatrix(): Isolating a Transform

Concept: Wrapping ONE shape's transformation in glPushMatrix()/glPopMatrix() prevents it from leaking into the next shape drawn.

Expected output: Two squares: the left one rotated 45 degrees, the right one NOT rotated, proving the push/pop isolation worked.

```
#define GL_SILENCE_DEPRECATION
#ifdef __APPLE_
#include <GLUT/glut.h>
#else
#include <GL/glew.h>
#include <GL/glut.h>
#endif
#include <iostream>
#include <cmath>
#include <cstdlib>
using namespace std;

#define BUFFER_OFFSET(i) ((void*)(i))
```

```
// glPushMatrix()/glPopMatrix() save and restore the current transformation
// matrix, so a transformation applied to ONE shape doesn't leak into the next.
void display(){
    glClear(GL_COLOR_BUFFER_BIT);

    glMatrixMode(GL_MODELVIEW);
    glLoadIdentity();

    // Shape 1: rotated, inside a push/pop pair
    glPushMatrix();
        glTranslatef(-0.4f, 0.0f, 0.0f);
        glRotatef(45.0f, 0.0f, 0.0f, 1.0f);
        glColor3f(0.9f, 0.3f, 0.1f);
        glBegin(GL_QUADS);
            glVertex2f(-0.2f, -0.2f);
            glVertex2f( 0.2f, -0.2f);
            glVertex2f( 0.2f,  0.2f);
            glVertex2f(-0.2f,  0.2f);
        glEnd();
    glPopMatrix();

    // Shape 2: NOT rotated -- the matrix was restored by glPopMatrix()
    glTranslatef(0.4f, 0.0f, 0.0f);
    glColor3f(0.1f, 0.5f, 0.9f);
    glBegin(GL_QUADS);
        glVertex2f(-0.2f, -0.2f);
        glVertex2f( 0.2f, -0.2f);
        glVertex2f( 0.2f,  0.2f);
        glVertex2f(-0.2f,  0.2f);
    glEnd();

    glFlush();
}

int main(int argc, char** argv){
    glutInit(&argc, argv);
    glutInitWindowSize(800, 600);
    glutInitDisplayMode(GLUT_RGBA);
    glutCreateWindow("simple");
    glutDisplayFunc(display);

#ifndef __APPLE_
    GLenum err = glewInit();
    if(err == GLEW_OK){
        glutMainLoop();
    }
#else
    glutMainLoop();
#endif
}
```

Example 06 — Transformation Order Matters

Concept: The SAME translate and rotate calls, written in opposite order, produce two visibly different results -- because OpenGL applies transforms in reverse of their written order.

Expected output: Two squares in different final positions/orientations, despite using the same two function calls.

```
#define GL_SILENCE_DEPRECATION
#ifdef __APPLE_
#include <GLUT/glut.h>
#else
#include <GL/glew.h>
#include <GL/glut.h>
#endif
#include <iostream>
#include <cmath>
#include <cstdlib>
using namespace std;

#define BUFFER_OFFSET(i) ((void*)(i))


// OpenGL applies transformations in REVERSE of the order they're written
// (each new matrix multiplies onto the current one). This example draws
// the SAME shape twice to show that translate-then-rotate is NOT the
// same as rotate-then-translate.
void display(){
    glClear(GL_COLOR_BUFFER_BIT);
    glMatrixMode(GL_MODELVIEW);

    // Left: rotate FIRST in code, so translate happens in the rotated frame
    glLoadIdentity();
    glTranslatef(-0.4f, 0.0f, 0.0f);
    glRotatef(45.0f, 0.0f, 0.0f, 1.0f);
    glColor3f(0.9f, 0.4f, 0.1f);
    glBegin(GL_QUADS);
        glVertex2f(-0.15f, -0.15f);
        glVertex2f( 0.15f, -0.15f);
        glVertex2f( 0.15f,  0.15f);
        glVertex2f(-0.15f,  0.15f);
    glEnd();

    // Right: translate FIRST in code, so rotation happens around the origin
    glLoadIdentity();
    glRotatef(45.0f, 0.0f, 0.0f, 1.0f);
    glTranslatef(0.4f, 0.0f, 0.0f);
    glColor3f(0.1f, 0.6f, 0.9f);
    glBegin(GL_QUADS);
        glVertex2f(-0.15f, -0.15f);
        glVertex2f( 0.15f, -0.15f);
```

```
        glVertex2f( 0.15f,  0.15f);
        glVertex2f(-0.15f,  0.15f);
    glEnd();

    glFlush();
}

int main(int argc, char** argv){
    glutInit(&argc, argv);
    glutInitWindowSize(800, 600);
    glutInitDisplayMode(GLUT_RGBA);
    glutCreateWindow("simple");
    glutDisplayFunc(display);

#ifndef __APPLE_
    GLenum err = glewInit();
    if(err == GLEW_OK){
        glutMainLoop();
    }
#else
    glutMainLoop();
#endif
}
```

Example 07 — glOrtho(): Orthographic Projection Setup

Concept: glOrtho(left, right, bottom, top, zNear, zFar) sets up a projection where object size does not change with depth.

Expected output: A green triangle rendered inside a -2..2 orthographic viewing box.

```
#define GL_SILENCE_DEPRECATION
#ifdef __APPLE_
#include <GLUT/glut.h>
#else
#include <GL/glew.h>
#include <GL/glut.h>
#endif
#include <iostream>
#include <cmath>
#include <cstdlib>
using namespace std;

#define BUFFER_OFFSET(i) ((void*)(i))


// glOrtho(left, right, bottom, top, zNear, zFar) sets up an orthographic
// (parallel) projection -- objects keep their size regardless of depth.
void init(){
```

```
    glMatrixMode(GL_PROJECTION);
    glLoadIdentity();
    glOrtho(-2.0, 2.0, -2.0, 2.0, -2.0, 2.0);
    glMatrixMode(GL_MODELVIEW);
}

void display(){
    glClear(GL_COLOR_BUFFER_BIT);
    glLoadIdentity();
    glColor3f(0.3f, 0.7f, 0.4f);
    glBegin(GL_TRIANGLES);
        glVertex2f( 0.0f,  1.0f);
        glVertex2f(-1.0f, -1.0f);
        glVertex2f( 1.0f, -1.0f);
    glEnd();
    glFlush();
}

int main(int argc, char** argv){
    glutInit(&argc, argv);
    glutInitWindowSize(800, 600);
    glutInitDisplayMode(GLUT_RGBA);
    glutCreateWindow("simple");
    glutDisplayFunc(display);

#ifndef __APPLE_
    GLenum err = glewInit();
    if(err == GLEW_OK){
        init();
        glutMainLoop();
    }
#else
        init();
    glutMainLoop();
#endif
}
```

Example 08 — gluOrtho2D(): Simplified 2D Projection

Concept: gluOrtho2D(left, right, bottom, top) is glOrtho with zNear/zFar automatically fixed at -1/1 -- ideal for pure 2D scenes.

Expected output: An orange quad rendered inside a -1..1 2D viewing box.

```
#define GL_SILENCE_DEPRECATION
#ifdef __APPLE_
#include <GLUT/glut.h>
#else
#include <GL/glew.h>
```

```
#include <GL/glut.h>
#endif
#include <iostream>
#include <cmath>
#include <cstdlib>
using namespace std;

#define BUFFER_OFFSET(i) ((void*)(i))


// gluOrtho2D(left, right, bottom, top) is a simplified glOrtho for pure
// 2D rendering -- it automatically sets zNear = -1 and zFar = 1.
void init(){
    glMatrixMode(GL_PROJECTION);
    glLoadIdentity();
    gluOrtho2D(-1.0, 1.0, -1.0, 1.0);
    glMatrixMode(GL_MODELVIEW);
}

void display(){
    glClear(GL_COLOR_BUFFER_BIT);
    glLoadIdentity();
    glColor3f(0.8f, 0.5f, 0.1f);
    glBegin(GL_QUADS);
        glVertex2f(-0.4f, -0.4f);
        glVertex2f( 0.4f, -0.4f);
        glVertex2f( 0.4f,  0.4f);
        glVertex2f(-0.4f,  0.4f);
    glEnd();
    glFlush();
}

int main(int argc, char** argv){
    glutInit(&argc, argv);
    glutInitWindowSize(800, 600);
    glutInitDisplayMode(GLUT_RGBA);
    glutCreateWindow("simple");
    glutDisplayFunc(display);

#ifndef __APPLE_
    GLenum err = glewInit();
    if(err == GLEW_OK){
        init();
        glutMainLoop();
    }
#else
        init();
    glutMainLoop();
#endif
}
```

Example 09 — Timer-Driven Rotation (animateTriangle Pattern)

Concept: The lecture's animateTriangle() pattern: a global angle variable increments every timer tick and wraps back to 0 after 360 degrees.

Expected output: A yellow triangle that continuously spins around the Y axis.

```
#define GL_SILENCE_DEPRECATION
#ifdef __APPLE_
#include <GLUT/glut.h>
#else
#include <GL/glew.h>
#include <GL/glut.h>
#endif
#include <iostream>
#include <cmath>
#include <cstdlib>
using namespace std;

#define BUFFER_OFFSET(i) ((void*)(i))


// The lecture's animateTriangle() pattern: a global angle variable is
// incremented every timer tick and wraps back to 0 after 360 degrees.
float angle = 1.0f;

void animateTriangle(int value){
    if(angle > 360) angle = 0;
    angle++;
    glutPostRedisplay();
    glutTimerFunc(50, animateTriangle, 1);
}

void display(){
    glClear(GL_COLOR_BUFFER_BIT);
    glMatrixMode(GL_MODELVIEW);
    glLoadIdentity();
    glRotatef(angle, 0.0f, 1.0f, 0.0f);

    glColor3f(0.9f, 0.9f, 0.1f);
    glBegin(GL_TRIANGLES);
        glVertex2f( 0.0f,  0.5f);
        glVertex2f(-0.5f, -0.5f);
        glVertex2f( 0.5f, -0.5f);
    glEnd();
    glFlush();
}

int main(int argc, char** argv){
    glutInit(&argc, argv);
    glutInitWindowSize(800, 600);
```

```
    glutInitDisplayMode(GLUT_RGBA);
    glutCreateWindow("simple");
    glutDisplayFunc(display);

#ifndef __APPLE_
    GLenum err = glewInit();
    if(err == GLEW_OK){
        glutTimerFunc(50, animateTriangle, 1);
        glutMainLoop();
    }
#else
        glutTimerFunc(50, animateTriangle, 1);
    glutMainLoop();
#endif
}
```

Example 10 — The Full Lecture Sample: VBO + Push/Pop + Timer

Concept: The lecture's exact triangle()/animateTriangle() program: a VBO set up once (position/scale applied once), rotated fresh every frame inside a glPushMatrix()/glPopMatrix() pair.

Expected output: A green triangle, positioned near the bottom of the screen, continuously rotating in place.

```
#define GL_SILENCE_DEPRECATION
#ifdef __APPLE_
#include <GLUT/glut.h>
#else
#include <GL/glew.h>
#include <GL/glut.h>
#endif
#include <iostream>
#include <cmath>
#include <cstdlib>
using namespace std;

#define BUFFER_OFFSET(i) ((void*)(i))


// The lecture's exact triangle()/animateTriangle() sample: a VBO
// triangle set up once, translated/scaled once, then rotated a little
// more every frame using glPushMatrix()/glPopMatrix() so the rotation
// doesn't accumulate onto other state.
bool initTriangle = true;
float angle = 1.0f;
GLuint VBOid;

GLfloat trianglevertices[] = {
```

```
     0.0f, .75f, 0.0f,
    -0.75f, 0.0f, 0.0f,
     0.75f, 0.0f, 0.0f
};

void triangle(){
    if(initTriangle){
        glGenBuffers(1, &VBOid);
        glBindBuffer(GL_ARRAY_BUFFER, VBOid);
        glBufferData(GL_ARRAY_BUFFER, sizeof(GLfloat)*9, trianglevertices,
GL_STATIC_DRAW);

        glEnableClientState(GL_VERTEX_ARRAY);

        glMatrixMode(GL_MODELVIEW);
        glLoadIdentity();

        glTranslatef(0.0f, -0.5f, 0.0f);
        glScalef(.75f, .75f, 0.0f);
        initTriangle = false;
    }

    glPushMatrix();
    glRotatef(angle, 0.0f, 0.0f, 1.0);

    glVertexPointer(3, GL_FLOAT, 0, 0);
    glDrawArrays(GL_TRIANGLES, 0, 3);
    glPopMatrix();
}

void animateTriangle(int value){
    angle++;
    glutPostRedisplay();
}

void display(){
    glClear(GL_COLOR_BUFFER_BIT);
    glColor4f(.16f, .72f, .08f, 1.0f);

    triangle();
    glFlush();

    glutTimerFunc(200, animateTriangle, 1);
}

int main(int argc, char** argv){
    glutInit(&argc, argv);
    glutInitWindowSize(800, 600);
    glutInitDisplayMode(GLUT_RGBA);
    glutCreateWindow("simple");
    glutDisplayFunc(display);
```

```
#ifndef __APPLE_
    GLenum err = glewInit();
    if(err == GLEW_OK){
        glutMainLoop();
    }
#else
    glutMainLoop();
#endif
}
```

Example 11 — gluPerspective(): Perspective Projection Setup

Concept: gluPerspective(fovy, aspect, zNear, zFar) creates a projection where farther objects appear smaller.

Expected output: A colored 3D triangle (a pyramid face), viewed with realistic perspective depth.

```
#define GL_SILENCE_DEPRECATION
#ifdef __APPLE_
#include <GLUT/glut.h>
#else
#include <GL/glew.h>
#include <GL/glut.h>
#endif
#include <iostream>
#include <cmath>
#include <cstdlib>
using namespace std;

#define BUFFER_OFFSET(i) ((void*)(i))


// gluPerspective(fovy, aspect, zNear, zFar) creates a perspective
// projection matrix -- objects farther from the viewer appear smaller.
void init(){
    glMatrixMode(GL_PROJECTION);
    glLoadIdentity();
    gluPerspective(60, 800.0/600.0, 1.0, 10.0);
    glMatrixMode(GL_MODELVIEW);
}

void display(){
    glClear(GL_COLOR_BUFFER_BIT);
    glLoadIdentity();
    glTranslatef(0.0f, 0.0f, -4.0f);

    glBegin(GL_TRIANGLES);
        glColor3f(1.0f, 0.0f, 0.0f);
```

```
        glVertex3f( 0.0f,  1.0f, 0.0f);
        glColor3f(0.0f, 1.0f, 0.0f);
        glVertex3f(-1.0f, -1.0f, 1.0f);
        glColor3f(0.0f, 0.0f, 1.0f);
        glVertex3f( 1.0f, -1.0f, 1.0f);
    glEnd();

    glFlush();
}

int main(int argc, char** argv){
    glutInit(&argc, argv);
    glutInitWindowSize(800, 600);
    glutInitDisplayMode(GLUT_RGBA);
    glutCreateWindow("simple");
    glutDisplayFunc(display);

#ifndef __APPLE_
    GLenum err = glewInit();
    if(err == GLEW_OK){
        init();
        glutMainLoop();
    }
#else
        init();
    glutMainLoop();
#endif
}
```

Example 12 — glFrustum(): Manual Perspective Control

Concept: glFrustum(left, right, bottom, top, zNear, zFar) manually defines the viewing frustum, offering more control than gluPerspective.

Expected output: A purple quad, viewed through a manually specified perspective frustum.

```
#define GL_SILENCE_DEPRECATION
#ifdef __APPLE_
#include <GLUT/glut.h>
#else
#include <GL/glew.h>
#include <GL/glut.h>
#endif
#include <iostream>
#include <cmath>
#include <cstdlib>
using namespace std;

#define BUFFER_OFFSET(i) ((void*)(i))
```

```
// glFrustum(left, right, bottom, top, zNear, zFar) manually defines the
// viewing frustum, offering more control than gluPerspective.
void init(){
    glMatrixMode(GL_PROJECTION);
    glLoadIdentity();
    glFrustum(-1.0, 1.0, -1.0, 1.0, 1.0, 10.0);
    glMatrixMode(GL_MODELVIEW);
}

void display(){
    glClear(GL_COLOR_BUFFER_BIT);
    glLoadIdentity();
    glTranslatef(0.0f, 0.0f, -3.0f);

    glColor3f(0.6f, 0.3f, 0.9f);
    glBegin(GL_QUADS);
        glVertex3f(-0.5f, -0.5f, 0.0f);
        glVertex3f( 0.5f, -0.5f, 0.0f);
        glVertex3f( 0.5f,  0.5f, 0.0f);
        glVertex3f(-0.5f,  0.5f, 0.0f);
    glEnd();

    glFlush();
}

int main(int argc, char** argv){
    glutInit(&argc, argv);
    glutInitWindowSize(800, 600);
    glutInitDisplayMode(GLUT_RGBA);
    glutCreateWindow("simple");
    glutDisplayFunc(display);

#ifndef __APPLE__
    GLenum err = glewInit();
    if(err == GLEW_OK){
        init();
        glutMainLoop();
    }
#else
        init();
    glutMainLoop();
#endif
}
```

Example 13 — gluLookAt(): Positioning the Camera

Concept: gluLookAt(eye..., center..., up...) defines where the camera is, what it's looking at, and which way is up.

Expected output: An orange quad, viewed from a camera positioned at (0,0,4) looking at the origin.

```
#define GL_SILENCE_DEPRECATION
#ifdef __APPLE_
#include <GLUT/glut.h>
#else
#include <GL/glew.h>
#include <GL/glut.h>
#endif
#include <iostream>
#include <cmath>
#include <cstdlib>
using namespace std;

#define BUFFER_OFFSET(i) ((void*)(i))



// gluLookAt(eyeX,eyeY,eyeZ, centerX,centerY,centerZ, upX,upY,upZ) defines
// a viewing transformation -- where the "camera" is and what it looks at.
void init(){
    glMatrixMode(GL_PROJECTION);
    glLoadIdentity();
    gluPerspective(60, 800.0/600.0, 1.0, 10.0);
    glMatrixMode(GL_MODELVIEW);
}

void display(){
    glClear(GL_COLOR_BUFFER_BIT);
    glLoadIdentity();

    // eye at (0,0,4), looking at the origin, with +Y as "up"
    gluLookAt(0.0, 0.0, 4.0, 0.0, 0.0, 0.0, 0.0, 1.0, 0.0);

    glColor3f(0.9f, 0.6f, 0.1f);
    glBegin(GL_QUADS);
        glVertex3f(-0.5f, -0.5f, 0.0f);
        glVertex3f( 0.5f, -0.5f, 0.0f);
        glVertex3f( 0.5f,  0.5f, 0.0f);
        glVertex3f(-0.5f,  0.5f, 0.0f);
    glEnd();

    glFlush();
}

int main(int argc, char** argv){
    glutInit(&argc, argv);
    glutInitWindowSize(800, 600);
    glutInitDisplayMode(GLUT_RGBA);
    glutCreateWindow("simple");
    glutDisplayFunc(display);
```

```
#ifndef __APPLE_
    GLenum err = glewInit();
    if(err == GLEW_OK){
        init();
        glutMainLoop();
    }
#else
        init();
    glutMainLoop();
#endif
}
```

Example 14 — The Full Lecture Capstone: Animated Color Pyramid

Concept: The complete lecture sample: a color pyramid built from ONE VBO (position + color via glBufferSubData), drawn with a glArrayElement() loop, viewed via gluPerspective + gluLookAt, rotated continuously with glPushMatrix()/glPopMatrix().

Expected output: A rotating, multi-colored pyramid, viewed in full 3D perspective.

```
#define GL_SILENCE_DEPRECATION
#ifdef __APPLE_
#include <GLUT/glut.h>
#else
#include <GL/glew.h>
#include <GL/glut.h>
#endif
#include <iostream>
#include <cmath>
#include <cstdlib>
using namespace std;

#define BUFFER_OFFSET(i) ((void*)(i))


// The full lecture sample program: a color pyramid (6 vertex "sides",
// each 3 verts x 4 faces = 12 total), position + color combined into
// ONE VBO via glBufferSubData, drawn with a glArrayElement() loop,
// rotated continuously with gluPerspective + gluLookAt set up in init().
GLuint VBOid;
bool initTriangle = true;
float angle = 1.0f;

GLfloat vertices[] = {
    //------------------- Triangle (4 faces of a pyramid)
     0.0f, 1.0f, 0.0f,
    -1.0f,-1.0f, 1.0f,
     1.0f,-1.0f, 1.0f,
```

```
     0.0f, 1.0f, 0.0f,
     1.0f,-1.0f, 1.0f,
     1.0f,-1.0f,-1.0f,

     0.0f, 1.0f, 0.0f,
     1.0f,-1.0f,-1.0f,
    -1.0f,-1.0f,-1.0f,

     0.0f, 1.0f, 0.0f,
    -1.0f,-1.0f,-1.0f,
    -1.0f,-1.0f, 1.0f,
};

GLfloat colorstriangle[] = {
    1.0f, 0.0f, 0.0f,
    0.0f, 1.0f, 0.0f,
    0.0f, 0.0f, 1.0f,

    1.0f, 0.0f, 0.0f,
    0.0f, 0.0f, 1.0f,
    0.0f, 1.0f, 0.0f,

    1.0f, 0.0f, 0.0f,
    0.0f, 1.0f, 0.0f,
    0.0f, 0.0f, 1.0f,

    1.0f,0.0f,0.0f,
    0.0f,0.0f,1.0f,
    0.0f,1.0f,0.0f,
};

void init(){
    glMatrixMode(GL_PROJECTION);
    glLoadIdentity();
    gluPerspective(60, 800.0/600.0, 1.0, 10.0);
}

void displayTriangle(){
    if(initTriangle){
        glGenBuffers(1, &VBOid);
        glBindBuffer(GL_ARRAY_BUFFER, VBOid);

        glBufferData(GL_ARRAY_BUFFER, ((9*4) + (9*4)) * sizeof(GLfloat), vertices,
GL_DYNAMIC_DRAW);
        glBufferSubData(GL_ARRAY_BUFFER, (9*4) * sizeof(GLfloat), (9*4) *
sizeof(GLfloat), colorstriangle);

        glEnableClientState(GL_VERTEX_ARRAY);
        glEnableClientState(GL_COLOR_ARRAY);
```

```
        glVertexPointer(3, GL_FLOAT, 0, 0);
        glColorPointer(3, GL_FLOAT, 0, BUFFER_OFFSET((9*4)*sizeof(GLfloat)));

        initTriangle = false;
    }

    glMatrixMode(GL_MODELVIEW);
    glLoadIdentity();
    gluLookAt(0.0,0.0,4.0, 0.0,0.0,0.0, 0.0,1.0,0.0);

    glPushMatrix();
    glRotatef(angle, 0.0f, 1.0f, 0.0f);

    glBegin(GL_TRIANGLES); // Displaying four sides
        for(int i = 0; i < 12; i++){
            glArrayElement(i);
        }
    glEnd();

    glFlush();
    glPopMatrix();
}

void animateTriangle(int value){
    if(angle > 360) angle = 0;
    angle++;
    glutPostRedisplay();
}

void display(){
    glClear(GL_COLOR_BUFFER_BIT);
    displayTriangle();
    glutTimerFunc(50, animateTriangle, 1);
}

int main(int argc, char** argv){
    glutInit(&argc, argv);
    glutInitWindowSize(800, 600);
    glutInitDisplayMode(GLUT_RGBA | GLUT_DEPTH);
    glutCreateWindow("simple");
    glutDisplayFunc(display);

#ifndef __APPLE_
    GLenum err = glewInit();
    if(err == GLEW_OK){
        init();
        glutMainLoop();
    }
#else
        init();
    glutMainLoop();
```

```
#endif
}
```

Example 15 — GLUT_DEPTH and the Z-Buffer

Concept: GLUT_DEPTH allocates a Z-buffer for hidden-surface elimination; toggling GL_DEPTH_TEST with the 'd' key shows what breaks without it.

Expected output: A near orange quad correctly drawn in front of a far blue quad; pressing 'd' disables depth testing and the draw order becomes visibly wrong.

```
#define GL_SILENCE_DEPRECATION
#ifdef __APPLE_
#include <GLUT/glut.h>
#else
#include <GL/glew.h>
#include <GL/glut.h>
#endif
#include <iostream>
#include <cmath>
#include <cstdlib>
using namespace std;

#define BUFFER_OFFSET(i) ((void*)(i))


// GLUT_DEPTH allocates a Z-buffer for hidden surface elimination, and
// glEnable(GL_DEPTH_TEST) turns depth testing on. This example draws a
// near quad and a far quad; toggle depth testing with the 'd' key to see
// the difference in which one appears "in front."
bool depthOn = true;

void display(){
    glClear(GL_COLOR_BUFFER_BIT | GL_DEPTH_BUFFER_BIT);
    if(depthOn) glEnable(GL_DEPTH_TEST); else glDisable(GL_DEPTH_TEST);

    glMatrixMode(GL_MODELVIEW);
    glLoadIdentity();
    glTranslatef(0.0f, 0.0f, -3.0f);

    // Far quad (drawn first, deeper Z)
    glColor3f(0.2f, 0.4f, 0.9f);
    glBegin(GL_QUADS);
        glVertex3f(-0.6f, -0.6f, -1.0f);
        glVertex3f( 0.6f, -0.6f, -1.0f);
        glVertex3f( 0.6f,  0.6f, -1.0f);
        glVertex3f(-0.6f,  0.6f, -1.0f);
    glEnd();
```

```
    // Near quad (drawn second, but SHOULD appear on top only if depth test is on)
    glColor3f(0.9f, 0.6f, 0.1f);
    glBegin(GL_QUADS);
        glVertex3f(-0.3f, -0.3f, 0.5f);
        glVertex3f( 0.3f, -0.3f, 0.5f);
        glVertex3f( 0.3f,  0.3f, 0.5f);
        glVertex3f(-0.3f,  0.3f, 0.5f);
    glEnd();

    glFlush();
}

void init(){
    glMatrixMode(GL_PROJECTION);
    glLoadIdentity();
    gluPerspective(60, 800.0/600.0, 1.0, 10.0);
    glMatrixMode(GL_MODELVIEW);
}

void monitorKeyboard(unsigned char key, int x, int y){
    if(key == 'd'){
        depthOn = !depthOn;
        glutPostRedisplay();
    }
}

int main(int argc, char** argv){
    glutInit(&argc, argv);
    glutInitWindowSize(800, 600);
    glutInitDisplayMode(GLUT_RGBA | GLUT_DEPTH);
    glutCreateWindow("simple");
    glutDisplayFunc(display);
    glutKeyboardFunc(monitorKeyboard);

#ifndef __APPLE_
    GLenum err = glewInit();
    if(err == GLEW_OK){
        init();
        glutMainLoop();
    }
#else
        init();
    glutMainLoop();
#endif
}
```

Example 16 — Hierarchical Modeling: a Two-Segment Arm

Concept: A base segment and an attached arm segment, where the arm inherits the base's rotation AND has its own additional local rotation, using nested glPushMatrix()/glPopMatrix().

Expected output: A blue base segment at an angle, with an orange arm segment attached at its end, angled further relative to the base.

```
#define GL_SILENCE_DEPRECATION
#ifdef __APPLE_
#include <GLUT/glut.h>
#else
#include <GL/glew.h>
#include <GL/glut.h>
#endif
#include <iostream>
#include <cmath>
#include <cstdlib>
using namespace std;

#define BUFFER_OFFSET(i) ((void*)(i))


// A hierarchical model: a base quad, and an "arm" quad attached to it.
// Rotating the base rotates the arm too, but glPushMatrix()/glPopMatrix()
// let us give the arm its OWN additional local rotation on top of that.
float baseAngle = 20.0f;
float armAngle  = 40.0f;

void display(){
    glClear(GL_COLOR_BUFFER_BIT);
    glMatrixMode(GL_MODELVIEW);
    glLoadIdentity();

    glPushMatrix();
        glTranslatef(0.0f, -0.4f, 0.0f);
        glRotatef(baseAngle, 0.0f, 0.0f, 1.0f);

        // Base segment
        glColor3f(0.3f, 0.6f, 0.9f);
        glBegin(GL_QUADS);
            glVertex2f(-0.1f, 0.0f);
            glVertex2f( 0.1f, 0.0f);
            glVertex2f( 0.1f, 0.5f);
            glVertex2f(-0.1f, 0.5f);
        glEnd();

        // Arm segment: inherits the base's transform, PLUS its own extra rotation
        glPushMatrix();
            glTranslatef(0.0f, 0.5f, 0.0f);
            glRotatef(armAngle, 0.0f, 0.0f, 1.0f);
            glColor3f(0.9f, 0.4f, 0.2f);
            glBegin(GL_QUADS);
```

```
                glVertex2f(-0.08f, 0.0f);
                glVertex2f( 0.08f, 0.0f);
                glVertex2f( 0.08f, 0.4f);
                glVertex2f(-0.08f, 0.4f);
            glEnd();
        glPopMatrix();
    glPopMatrix();

    glFlush();
}

int main(int argc, char** argv){
    glutInit(&argc, argv);
    glutInitWindowSize(800, 600);
    glutInitDisplayMode(GLUT_RGBA);
    glutCreateWindow("simple");
    glutDisplayFunc(display);

#ifndef __APPLE_
    GLenum err = glewInit();
    if(err == GLEW_OK){
        glutMainLoop();
    }
#else
    glutMainLoop();
#endif
}
```

Example 17 — Three Independent Objects, Three Push/Pop Pairs

Concept: Three copies of the same triangle, each with its OWN glPushMatrix()/glPopMatrix() pair, so each object's transform never leaks into the next.

Expected output: Three triangles in a row: one plain, one rotated 90 degrees, one scaled 1.5x -- each independently transformed.

```
#define GL_SILENCE_DEPRECATION
#ifdef __APPLE_
#include <GLUT/glut.h>
#else
#include <GL/glew.h>
#include <GL/glut.h>
#endif
#include <iostream>
#include <cmath>
#include <cstdlib>
using namespace std;

#define BUFFER_OFFSET(i) ((void*)(i))
```

```
// Three independent copies of the same shape, each with its OWN
// glPushMatrix()/glPopMatrix() pair, so each object's transform never
// leaks into the next one's.
void drawTriangle(){
    glBegin(GL_TRIANGLES);
        glVertex2f( 0.0f,  0.15f);
        glVertex2f(-0.15f, -0.15f);
        glVertex2f( 0.15f, -0.15f);
    glEnd();
}

void display(){
    glClear(GL_COLOR_BUFFER_BIT);
    glMatrixMode(GL_MODELVIEW);
    glLoadIdentity();

    glPushMatrix();
        glTranslatef(-0.6f, 0.0f, 0.0f);
        glRotatef(0.0f, 0.0f, 0.0f, 1.0f);
        glColor3f(0.9f, 0.2f, 0.2f);
        drawTriangle();
    glPopMatrix();

    glPushMatrix();
        glTranslatef(0.0f, 0.0f, 0.0f);
        glRotatef(90.0f, 0.0f, 0.0f, 1.0f);
        glColor3f(0.2f, 0.9f, 0.2f);
        drawTriangle();
    glPopMatrix();

    glPushMatrix();
        glTranslatef(0.6f, 0.0f, 0.0f);
        glScalef(1.5f, 1.5f, 1.0f);
        glColor3f(0.2f, 0.2f, 0.9f);
        drawTriangle();
    glPopMatrix();

    glFlush();
}

int main(int argc, char** argv){
    glutInit(&argc, argv);
    glutInitWindowSize(800, 600);
    glutInitDisplayMode(GLUT_RGBA);
    glutCreateWindow("simple");
    glutDisplayFunc(display);

#ifndef __APPLE__
    GLenum err = glewInit();
```

```
    if(err == GLEW_OK){
        glutMainLoop();
    }
#else
    glutMainLoop();
#endif
}
```

Example 18 — glViewport(): Split-Screen Rendering

Concept: Calling glViewport() twice with different regions splits an 800x600 window into two independently drawn halves.

Expected output: The same green triangle drawn twice, once in the left half of the window and once in the right half.

```
#define GL_SILENCE_DEPRECATION
#ifdef __APPLE_
#include <GLUT/glut.h>
#else
#include <GL/glew.h>
#include <GL/glut.h>
#endif
#include <iostream>
#include <cmath>
#include <cstdlib>
using namespace std;

#define BUFFER_OFFSET(i) ((void*)(i))


// glViewport(x, y, width, height) defines the drawable region of the
// window. This example splits an 800x600 window into two side-by-side
// viewports, each drawing the SAME triangle.
void drawTriangle(){
    glColor3f(0.4f, 0.8f, 0.3f);
    glBegin(GL_TRIANGLES);
        glVertex2f( 0.0f,  0.5f);
        glVertex2f(-0.5f, -0.5f);
        glVertex2f( 0.5f, -0.5f);
    glEnd();
}

void display(){
    glClear(GL_COLOR_BUFFER_BIT);

    // Left half of the window
    glViewport(0, 0, 400, 600);
```

```
    drawTriangle();

    // Right half of the window
    glViewport(400, 0, 400, 600);
    drawTriangle();

    glFlush();
}

int main(int argc, char** argv){
    glutInit(&argc, argv);
    glutInitWindowSize(800, 600);
    glutInitDisplayMode(GLUT_RGBA);
    glutCreateWindow("simple");
    glutDisplayFunc(display);

#ifndef __APPLE_
    GLenum err = glewInit();
    if(err == GLEW_OK){
        glutMainLoop();
    }
#else
    glutMainLoop();
#endif
}
```

Example 19 — glViewport() Tied to Window Resizing

Concept: glutReshapeFunc() lets a program update glViewport() (and the projection) whenever the window is resized, keeping the rendering area correct.

Expected output: A purple triangle that stays correctly proportioned no matter how the window is resized.

```
#define GL_SILENCE_DEPRECATION
#ifdef __APPLE_
#include <GLUT/glut.h>
#else
#include <GL/glew.h>
#include <GL/glut.h>
#endif
#include <iostream>
#include <cmath>
#include <cstdlib>
using namespace std;

#define BUFFER_OFFSET(i) ((void*)(i))
```

```
// glViewport is usually the same as the window size, but it needs to be
// updated whenever the window is RESIZED. glutReshapeFunc lets us hook
// that event and call glViewport with the new dimensions.
void display(){
    glClear(GL_COLOR_BUFFER_BIT);
    glColor3f(0.7f, 0.3f, 0.8f);
    glBegin(GL_TRIANGLES);
        glVertex2f( 0.0f,  0.5f);
        glVertex2f(-0.5f, -0.5f);
        glVertex2f( 0.5f, -0.5f);
    glEnd();
    glFlush();
}

void reshape(int width, int height){
    glViewport(0, 0, width, height);
    glMatrixMode(GL_PROJECTION);
    glLoadIdentity();
    gluOrtho2D(-1.0, 1.0, -1.0, 1.0);
    glMatrixMode(GL_MODELVIEW);
}

int main(int argc, char** argv){
    glutInit(&argc, argv);
    glutInitWindowSize(800, 600);
    glutInitDisplayMode(GLUT_RGBA);
    glutCreateWindow("simple");
    glutDisplayFunc(display);
    glutReshapeFunc(reshape);

#ifndef __APPLE__
    GLenum err = glewInit();
    if(err == GLEW_OK){
        glutMainLoop();
    }
#else
    glutMainLoop();
#endif
}
```

Example 20 — Capstone: Hierarchical Animated Scene With Full Setup

Concept: A two-segment hierarchical arm (glPushMatrix/glPopMatrix) that both rotates as a whole and animates its second segment, viewed via gluPerspective + gluLookAt, with GLUT_DEPTH, an error callback, and ESC-key cleanup.

Expected output: A blue base segment rotating slowly, with an orange arm segment attached that rotates faster relative to it; pressing ESC exits the program.

```
#define GL_SILENCE_DEPRECATION
#ifdef __APPLE_
#include <GLUT/glut.h>
#else
#include <GL/glew.h>
#include <GL/glut.h>
#endif
#include <iostream>
#include <cmath>
#include <cstdlib>
using namespace std;

#define BUFFER_OFFSET(i) ((void*)(i))



// Capstone: a hierarchical two-segment "arm" (glPushMatrix()/glPopMatrix())
// that both rotates as a whole and animates its second segment, viewed
// through gluPerspective + gluLookAt, with GLUT_DEPTH enabled, an error
// callback registered, and ESC-key cleanup.
float wholeAngle = 0.0f;
float armAngle = 0.0f;

void GLAPIENTRY MessageCallback(GLenum source, GLenum type, GLuint id,
                                GLenum severity, GLsizei length,
                                const GLchar* message, const void* userParam){
    fprintf(stderr, "GL CALLBACK: %s type = 0x%x, severity = 0x%x, message = %s\n",
            (type == GL_DEBUG_TYPE_ERROR ? "** GL ERROR **" : ""),
            type, severity, message);
}

void init(){
    glMatrixMode(GL_PROJECTION);
    glLoadIdentity();
    gluPerspective(60, 800.0/600.0, 1.0, 10.0);
}

void display(){
    glClear(GL_COLOR_BUFFER_BIT | GL_DEPTH_BUFFER_BIT);
    glEnable(GL_DEPTH_TEST);

    glMatrixMode(GL_MODELVIEW);
    glLoadIdentity();
    gluLookAt(0.0,0.0,4.0, 0.0,0.0,0.0, 0.0,1.0,0.0);

    glPushMatrix();
        glRotatef(wholeAngle, 0.0f, 1.0f, 0.0f);

        // Base segment
        glColor3f(0.3f, 0.6f, 0.9f);
        glBegin(GL_QUADS);
            glVertex3f(-0.15f, -0.6f, 0.0f);
```

```
            glVertex3f( 0.15f, -0.6f, 0.0f);
            glVertex3f( 0.15f,  0.0f, 0.0f);
            glVertex3f(-0.15f,  0.0f, 0.0f);
        glEnd();

        glPushMatrix();
            glTranslatef(0.0f, 0.0f, 0.0f);
            glRotatef(armAngle, 0.0f, 0.0f, 1.0f);
            glColor3f(0.9f, 0.4f, 0.2f);
            glBegin(GL_QUADS);
                glVertex3f(-0.12f, 0.0f, 0.0f);
                glVertex3f( 0.12f, 0.0f, 0.0f);
                glVertex3f( 0.12f, 0.6f, 0.0f);
                glVertex3f(-0.12f, 0.6f, 0.0f);
            glEnd();
        glPopMatrix();
    glPopMatrix();

    glFlush();
}

void animate(int value){
    wholeAngle += 1.0f;
    armAngle += 2.0f;
    if(wholeAngle > 360) wholeAngle = 0;
    if(armAngle > 360) armAngle = 0;
    glutPostRedisplay();
    glutTimerFunc(50, animate, 0);
}

void monitorKeyboard(unsigned char key, int x, int y){
    if(key == 27){
        exit(0);
    }
}

int main(int argc, char** argv){
    glutInit(&argc, argv);
    glutInitWindowSize(800, 600);
    glutInitDisplayMode(GLUT_RGBA | GLUT_DEPTH);
    glutCreateWindow("simple");
    glutDisplayFunc(display);
    glutKeyboardFunc(monitorKeyboard);

#ifndef __APPLE_
    GLenum err = glewInit();
    if(err == GLEW_OK){
        glEnable(GL_DEBUG_OUTPUT);
        glDebugMessageCallback(MessageCallback, 0);
        init();
        glutTimerFunc(50, animate, 0);
        glutMainLoop();
```

```
    }
#else
        glEnable(GL_DEBUG_OUTPUT);
        glDebugMessageCallback(MessageCallback, 0);
        init();
        glutTimerFunc(50, animate, 0);
    glutMainLoop();
#endif
}
```

Part B — Practice Exercises (Q01–Q20)

Attempt each exercise on your own first. Compile and run your own solution, compare its output to the task description, and only THEN check Part C. Difficulty is marked Easy, Medium, or Hard. Each exercise's "Task" line is the exact question you must answer with your program.

Easy Exercises

Exercise Q01 — Translate a Square Upward [Easy]

Objective: Practice basic glTranslatef() usage on a simple shape.

Task (Question): Move a square UP by 0.4 units using glTranslatef(), starting from glLoadIdentity().

Hints:

- Always call glMatrixMode(GL_MODELVIEW) and glLoadIdentity() before your glTranslatef() call, so you start from a known state.
- Only the Y argument to glTranslatef() needs to be nonzero for a purely vertical move.

Exercise Q02 — Rotate Around the X Axis [Easy]

Objective: Practice glRotatef() around an axis OTHER than Z, in a 3D perspective scene.

Task (Question): Rotate a quad 60 degrees around the X axis (not Z) inside a gluPerspective scene, so the "flattening" effect of rotating away from the viewer is visible.

Hints:

- The axis argument to glRotatef() is (1,0,0) for the X axis, not (0,0,1).
- You'll need gluPerspective and a glTranslatef(0,0,-3) or similar to push the quad back into view first.

Exercise Q03 — Non-Uniform Scaling [Easy]

Objective: Practice glScalef() with DIFFERENT X and Y factors to turn a square into a rectangle.

Task (Question): Scale a square non-uniformly using glScalef() so it becomes a wide, short rectangle (different X and Y scale factors).

Hints:

- glScalef(2.0f, 0.5f, 1.0f) doubles the width while halving the height.
- The Z scale factor doesn't matter for a purely 2D shape, but convention is to leave it at 1.0f.

Exercise Q04 — Three Quads, One Rotated [Easy]

Objective: Practice using glPushMatrix()/glPopMatrix() to isolate ONE shape's transformation among several.

Task (Question): Draw three quads in a row, where only the MIDDLE one is rotated 45 degrees, using glPushMatrix()/glPopMatrix() to keep that rotation from affecting the other two.

Hints:

- Give each quad its own glPushMatrix()/glPopMatrix() pair, even the ones that aren't rotated -- it's good habit and keeps the code symmetric.
- Draw the quads in a fixed order (left, middle, right) so it's easy to tell which one is which.

Exercise Q05 — A Non-Symmetric glOrtho Viewing Box [Easy]

Objective: Practice glOrtho() with a viewing box that does NOT start at 0 or use symmetric bounds.

Task (Question): Set up glOrtho with a viewing box from 0 to 4 in X and 0 to 3 in Y (not the usual -1..1), and draw a shape positioned using coordinates that make sense in that range.

Hints:

- Your vertex coordinates now need to be somewhere between 0 and 4 (X) and 0 and 3 (Y) to be visible at all.
- Try picking a triangle roughly centered in that box, e.g., around (2, 1.5).

Exercise Q06 — gluOrtho2D With Pixel-Style Coordinates [Easy]

Objective: Practice gluOrtho2D() using window-pixel-style coordinates instead of the usual -1..1 range.

Task (Question): Set up gluOrtho2D using coordinates from 0 to 800 in X and 0 to 600 in Y (matching an 800x600 window), and draw a shape positioned using pixel-like numbers.

Hints:

- With this setup, a vertex at (400, 300) is now the CENTER of the window, not (0,0).
- This pixel-coordinate style is common in 2D games and UI code, where designers think in pixels.

Exercise Q07 — Animate a Rotating Quad [Easy]

Objective: Practice the lecture's animateTriangle() timer pattern on a shape of your own.

Task (Question): Animate a quad's rotation angle using a glutTimerFunc callback that increments the angle and wraps it back to 0 after 360 degrees, matching the lecture's animateTriangle() structure.

Hints:

- Call glutPostRedisplay() inside your timer callback so the rotation is actually visible.
- Reschedule the timer as the LAST line inside the callback itself, just like the lecture's example.

Medium Exercises

Exercise Q08 — Scale-Then-Translate vs. Translate-Then-Scale [Medium]

Objective: Practice directly comparing how transform ORDER changes the final result when scale and translate are combined.

Task (Question): Draw the SAME square twice: once with translate written before scale in the code, once with scale written before translate, and observe that the translated distance ends up different in each case.

Hints:

- When scale is applied AFTER translate (in code), the translation itself gets scaled too -- this is the key insight to watch for.
- Use very different colors for the two squares so it's obvious which is which.

Exercise Q09 — Three-Segment Hierarchical Chain [Medium]

Objective: Practice NESTED glPushMatrix()/glPopMatrix() pairs to build a chain more than two segments deep.

Task (Question): Build a three-segment hierarchical chain (base, middle, tip), where each segment has its own local rotation angle and each inherits ALL of its parents' transforms, using nested glPushMatrix()/glPopMatrix() pairs.

Hints:

- Each new segment should glTranslatef() to the END of its parent segment before applying its own rotation.
- Draw each segment a different color so the hierarchy is easy to see and verify visually.

Exercise Q10 — Comparing Wide vs. Narrow Field of View [Medium]

Objective: Practice adjusting gluPerspective's fovy parameter and observing the visual effect.

Task (Question): Set up a scene where pressing the 'v' key toggles between a WIDE field of view (90 degrees) and a NARROW field of view (30 degrees) via gluPerspective, redrawing the same shape at the same position.

Hints:

- You'll need to call gluPerspective again (inside display() or a helper) every time the toggle changes, since it affects the PROJECTION matrix.
- A wider fovy makes objects appear smaller/farther for the same actual distance -- a narrower fovy "zooms in".

Exercise Q11 — An Asymmetric glFrustum [Medium]

Objective: Practice glFrustum() with left/right bounds that are NOT mirror images of each other.

Task (Question): Build an asymmetric frustum using glFrustum() (e.g., left = -0.5, right = 1.5) and observe how this shifts the apparent center of the view compared to a symmetric frustum.

Hints:

- Symmetric frustums have left == -right; asymmetric frustums (used for things like stereo rendering) don't.
- Keep bottom/top symmetric and only change left/right, so it's easier to see just ONE effect at a time.

Exercise Q12 — Orbiting Camera via Animated gluLookAt [Medium]

Objective: Practice animating the EYE position passed to gluLookAt(), instead of rotating the object itself.

Task (Question): Animate the eye position in gluLookAt() so the camera orbits around a fixed quad (using sinf/cosf of an increasing angle), rather than rotating the quad with glRotatef().

Hints:

- Compute eyeX and eyeZ from sinf(angle) and cosf(angle) times a fixed radius, keeping eyeY constant for a flat circular orbit.
- The quad itself should NOT have any rotation applied -- only the camera moves.

Exercise Q13 — Static Color Quad via glBufferSubData + glArrayElement [Medium]

Objective: Practice the lecture's position+color-in-one-VBO pattern on a simpler, STATIC (non-animated) shape.

Task (Question): Build a color quad using ONE VBO (position data followed by color data via glBufferSubData), read out with a glArrayElement() loop, following the lecture's displayTriangle() pattern but without any rotation or timer.

Hints:

- Reserve the FULL combined size in your first glBufferData call, then use glBufferSubData for the color block.
- Loop glArrayElement() from 0 to 3 (4 vertices) inside a glBegin(GL_QUADS)/glEnd() pair.

Exercise Q14 — Depth Testing With Three Triangles [Medium]

Objective: Practice extending the depth-test toggle demo to MULTIPLE overlapping shapes at different depths.

Task (Question): Extend the depth-test comparison to THREE overlapping triangles at different Z depths, toggled on and off with the 'd' key, so students can see all three change draw order at once.

Hints:

- Give each triangle a noticeably different Z value (e.g., -0.5, 0.0, 0.5) so the depth differences are easy to see.
- Clear GL_DEPTH_BUFFER_BIT along with GL_COLOR_BUFFER_BIT every frame, or the depth buffer will keep stale values.

Hard Exercises

Exercise Q15 — Lecture Pyramid, Rotated on a Different Axis, With Cleanup [Hard]

Objective: Practice modifying the full lecture capstone pattern (rotation axis) AND adding proper ESC-key cleanup.

Task (Question): Rebuild the lecture's color pyramid example, but rotate it around the X axis instead of Y, and add a monitorKeyboard() ESC handler that releases the VBO before exit(0).

Hints:

- Only the axis argument to glRotatef() needs to change -- (1,0,0) instead of (0,1,0).
- Your cleanup handler should disable both client states, unbind the buffer, and call glDeleteBuffers, matching Module 5/6's pattern.

Exercise Q16 — Split Screen: Orthographic vs. Perspective, Side by Side [Hard]

Objective: Practice setting up TWO DIFFERENT projection types in two different viewports within the same window.

Task (Question): Split the window into two viewports showing the SAME 3D shape: orthographic projection on the left, perspective projection on the right, so the difference is directly comparable.

Hints:

- Each viewport needs its OWN glMatrixMode(GL_PROJECTION)/glLoadIdentity()/projection-setup call, since the projection matrix is shared across the whole context.
- Use the SAME shape and the SAME modelview transform for both halves, so the projection type is the only variable.

Exercise Q17 — Error Callback With an Intentional Trigger [Hard]

Objective: Practice registering glDebugMessageCallback AND deliberately triggering an error inside a projection setup function.

Task (Question): Register a glDebugMessageCallback, then intentionally call glEnable with an INVALID enum inside your projection setup function, so the callback fires and prints a diagnostic message before the rest of the scene renders normally.

Hints:

- The invalid glEnable() call doesn't need to do anything useful -- its only purpose is to trigger the callback for you to observe.
- Check your terminal/console output, not the graphics window, to see the callback's printed message.

Exercise Q18 — Keyboard-Controlled Two-Joint Arm [Hard]

Objective: Practice combining hierarchical glPushMatrix()/glPopMatrix() with interactive keyboard control of MULTIPLE independent angles.

Task (Question): Build a two-joint arm where the 'a'/'A' keys increase/decrease the base segment's angle and 's'/'S' keys increase/decrease the second joint's angle, using a glPushMatrix()/glPopMatrix() hierarchy so the second joint always inherits the base's current rotation.

Hints:

- Keep baseAngle and jointAngle as separate global floats, each adjusted independently by its own key pair.
- Call glutPostRedisplay() after every key press so the change is immediately visible.

Exercise Q19 — Two Viewports, Two Cameras, One Animated [Hard]

Objective: Practice running TWO independent gluLookAt cameras in two viewports simultaneously, one static and one animated.

Task (Question): Split the window into two viewports showing the SAME 3D scene from two different gluLookAt camera positions at once: the left viewport with a FIXED camera, the right viewport with an ANIMATED orbiting camera.

Hints:

- Each viewport needs its own full glMatrixMode(GL_PROJECTION)/gluPerspective AND glMatrixMode(GL_MODELVIEW)/gluLookAt setup.
- Only the RIGHT viewport's gluLookAt eye position should depend on your animated orbit angle.

Exercise Q20 — Capstone: Color VBO Base + Animated Hierarchical Arm [Hard]

Objective: Practice combining EVERY technique from this module into one program: a color VBO object, a hierarchical animated arm, full projection/viewing setup, an error callback, and cleanup.

Task (Question): Build a capstone scene: a FIXED color VBO base object (position+color via glBufferSubData, drawn with glArrayElement), with a hierarchical rotating "arm" (glPushMatrix/glPopMatrix, immediate mode) attached above it, both viewed via gluPerspective + gluLookAt, with GLUT_DEPTH, a registered error callback, and ESC-key cleanup that releases the VBO.

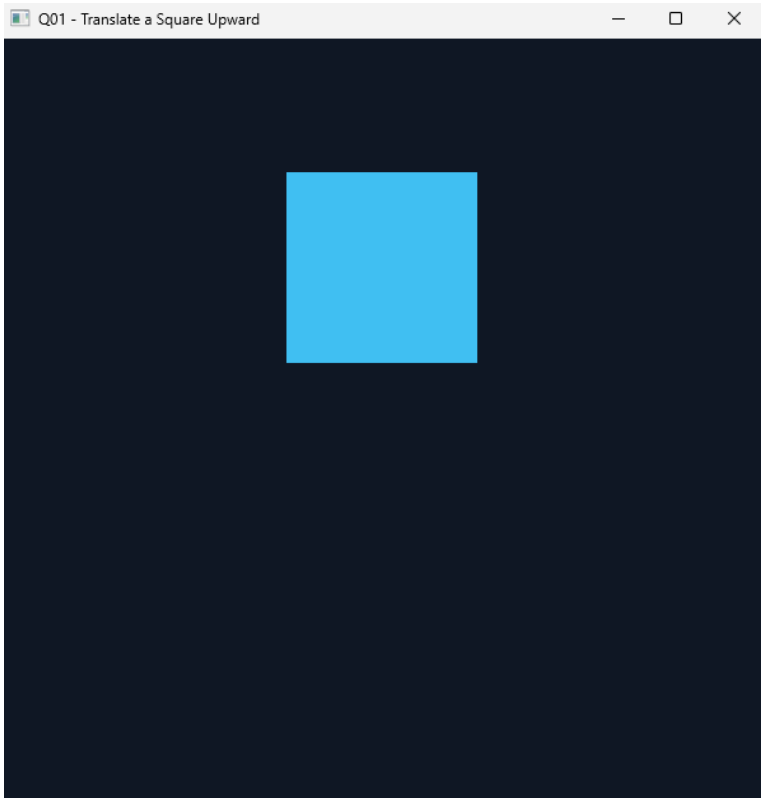
Hints:

- Disable GL_VERTEX_ARRAY/GL_COLOR_ARRAY client states before drawing the immediate-mode arm, then re-enable them if you draw the VBO base again later.
- Count your cleanup steps carefully: disable both client states, unbind the buffer, delete it, THEN exit(0).

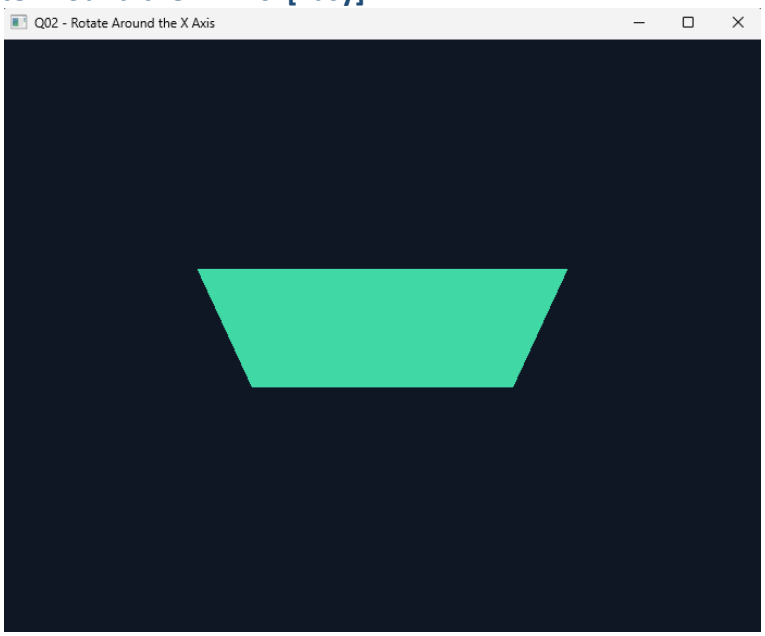
Part C — Outputs / Screen Shots (Ex01–Ex20)

Easy Exercises

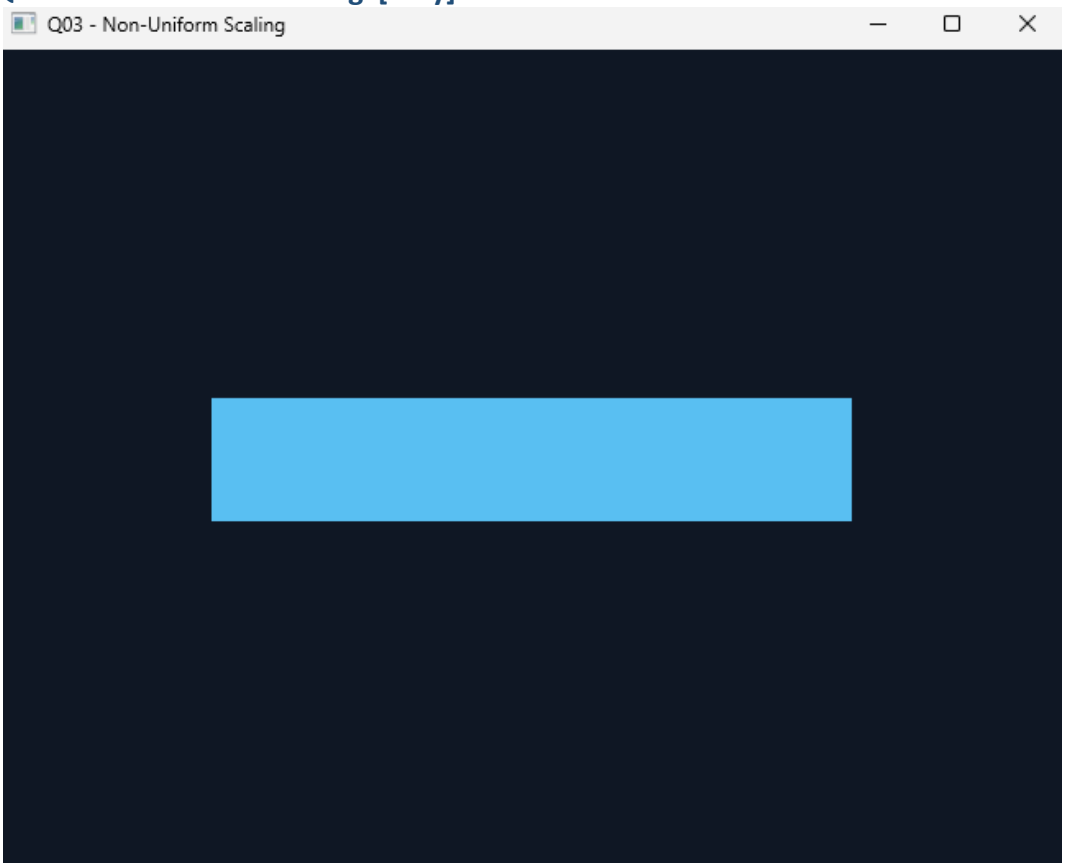
Exercise Q01 — Translate a Square Upward [Easy]



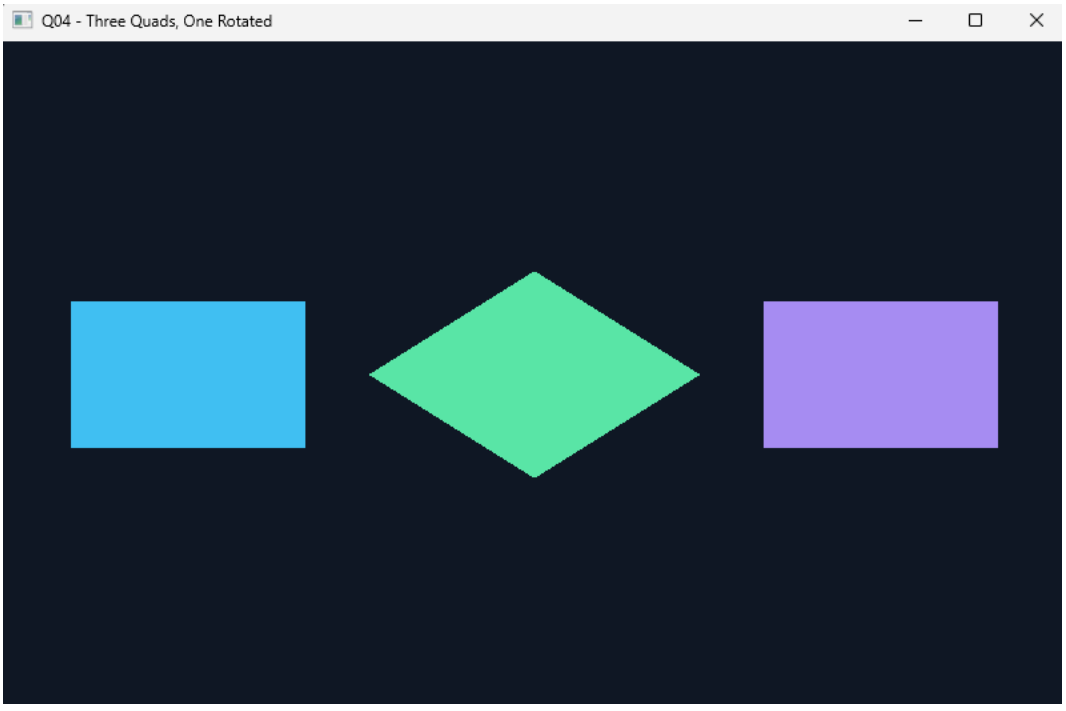
Exercise Q02 — Rotate Around the X Axis [Easy]



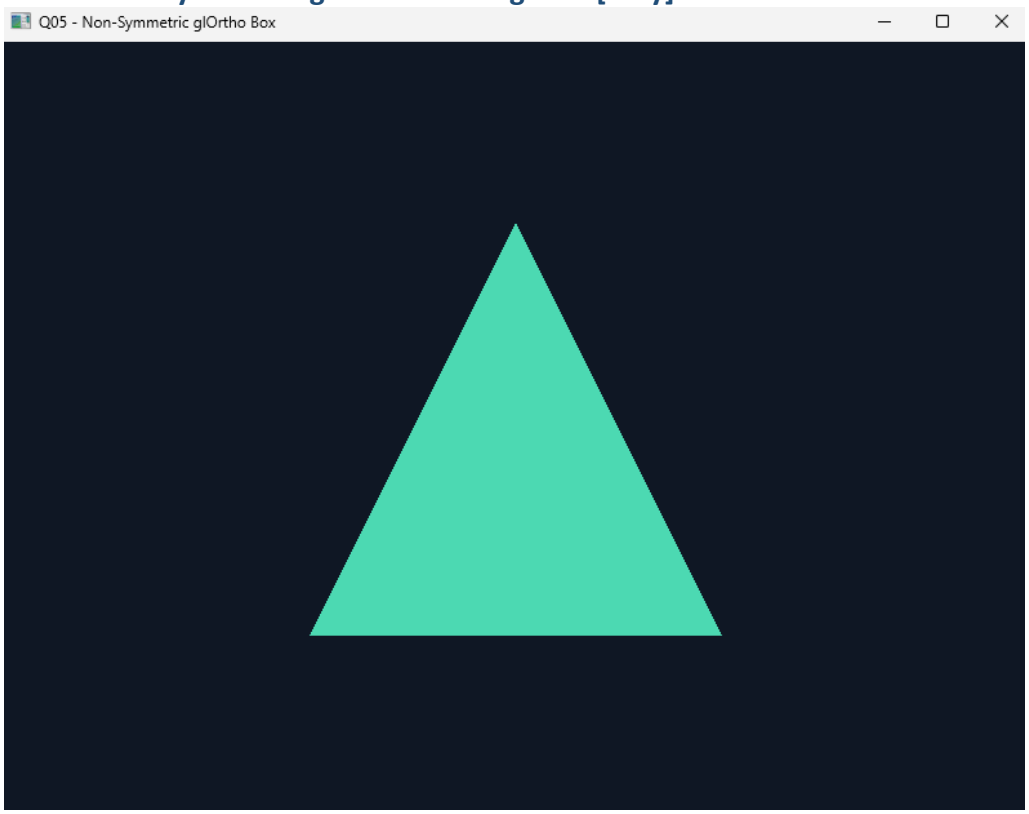
Exercise Q03 — Non-Uniform Scaling [Easy]



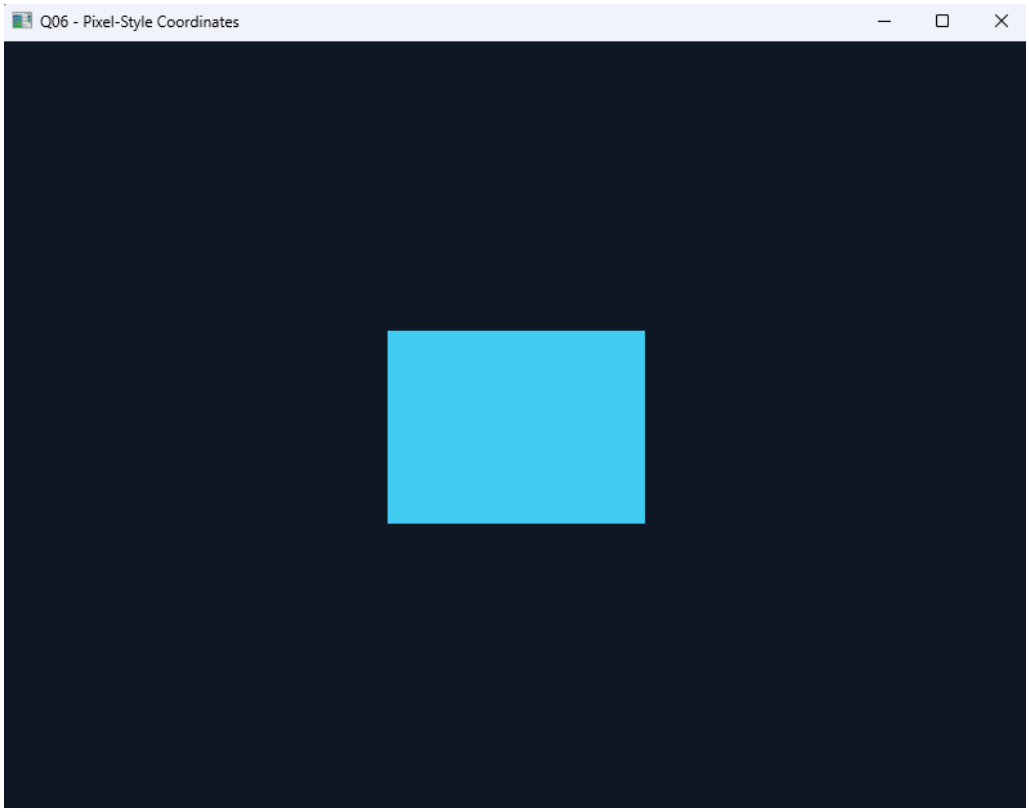
Exercise Q04 — Three Quads, One Rotated [Easy]



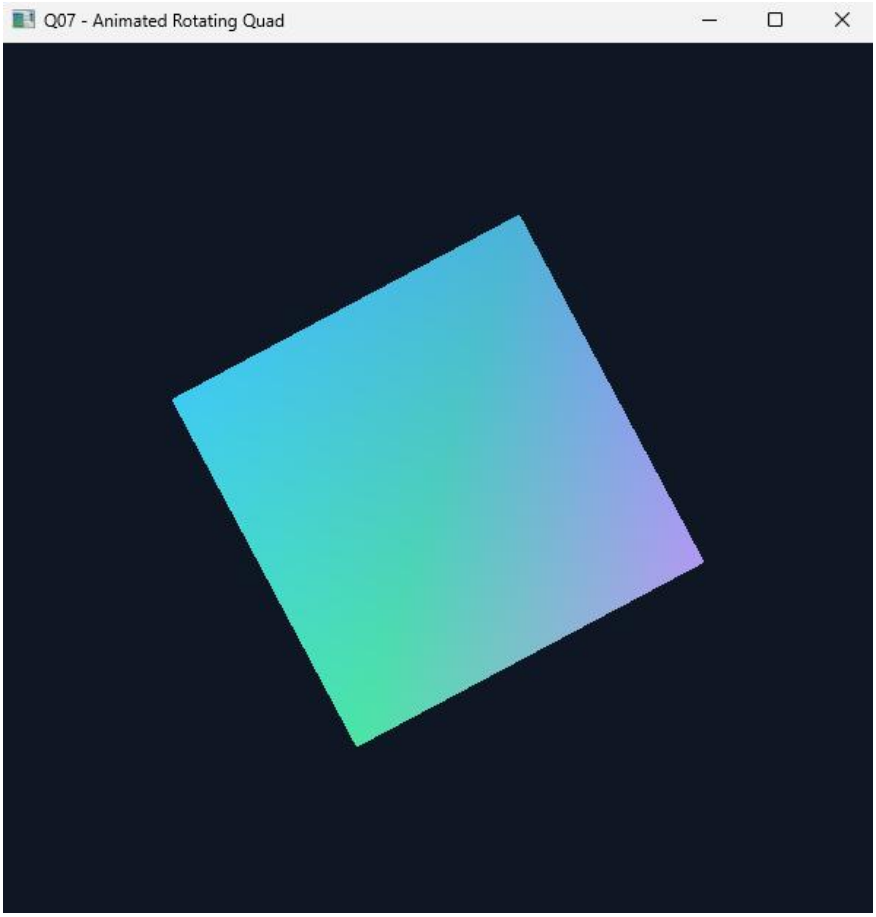
Exercise Q05 — A Non-Symmetric glOrtho Viewing Box [Easy]



Exercise Q06 — gluOrtho2D With Pixel-Style Coordinates [Easy]



Exercise Q07 — Animate a Rotating Quad [Easy]

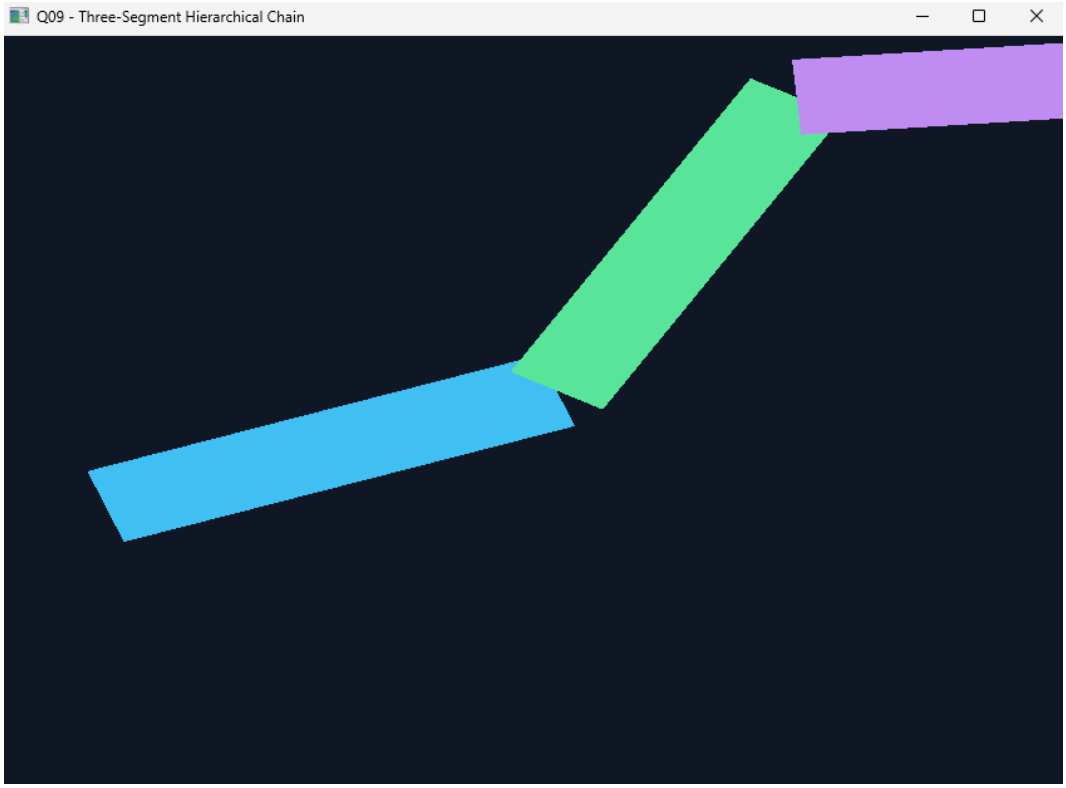


Medium Exercises

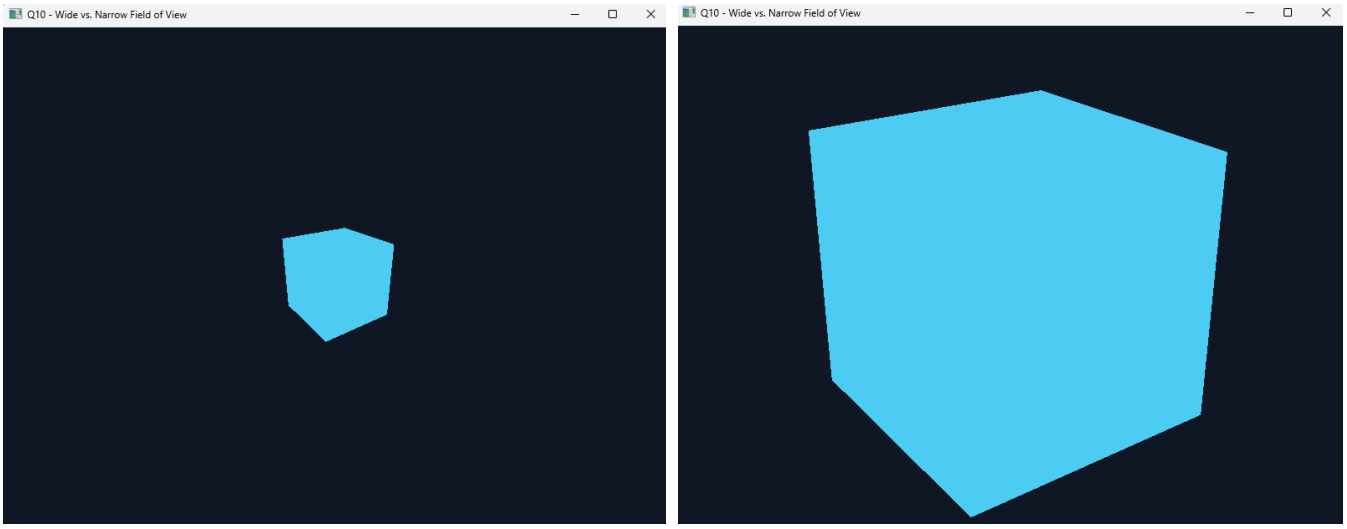
Exercise Q08 — Scale-Then-Translate vs. Translate-Then-Scale [Medium]



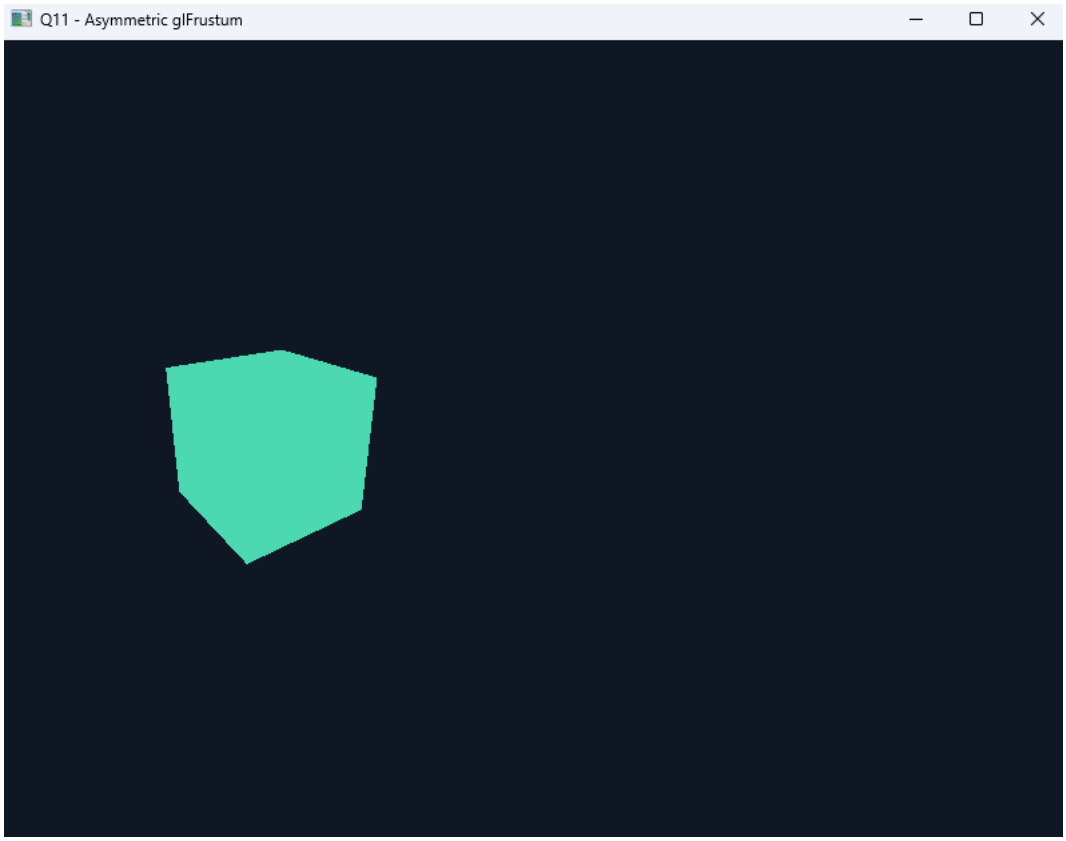
Exercise Q09 — Three-Segment Hierarchical Chain [Medium]



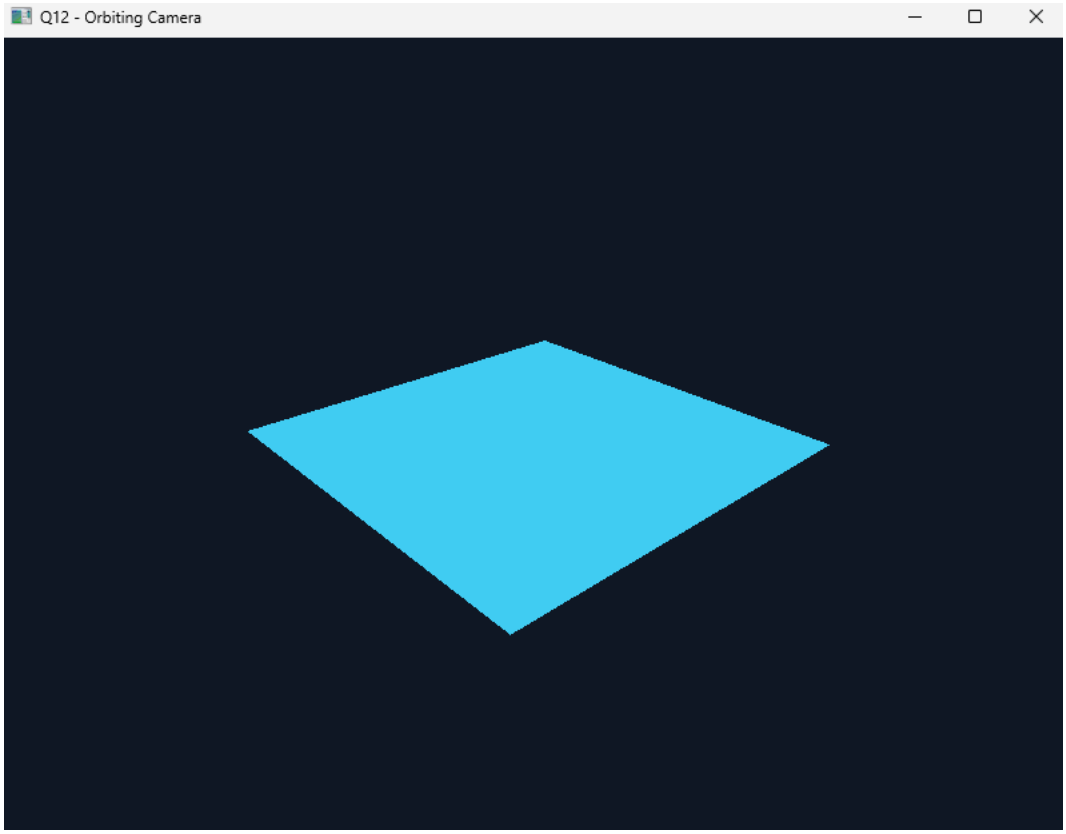
Exercise Q10 — Comparing Wide vs. Narrow Field of View [Medium]



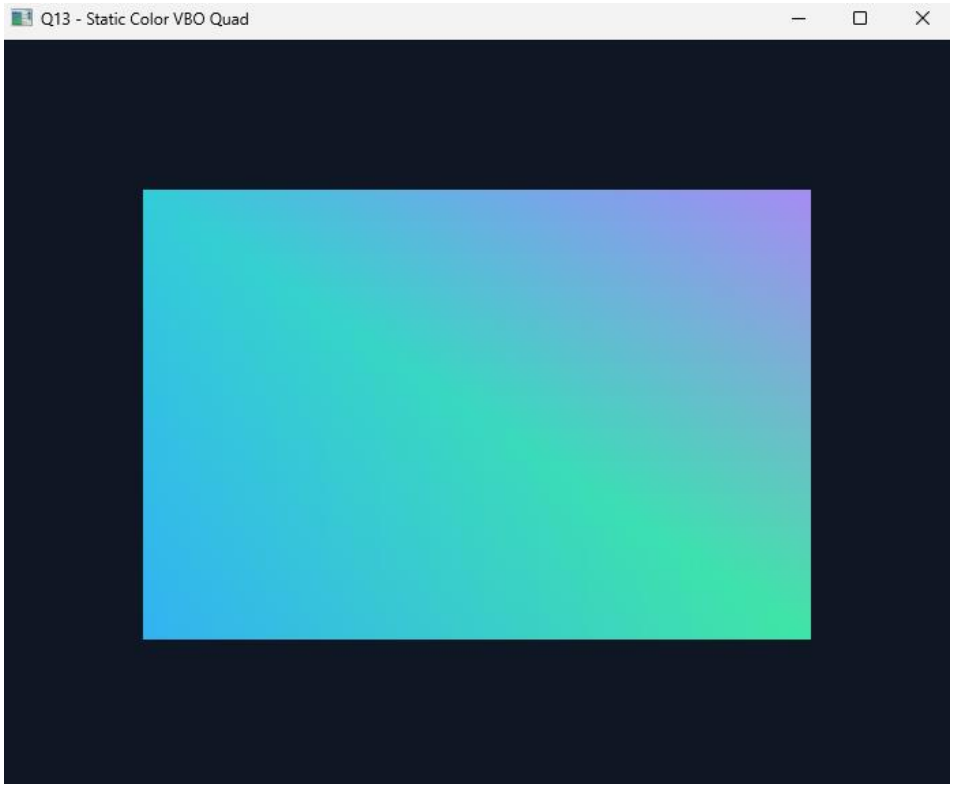
Exercise Q11 — An Asymmetric glFrustum [Medium]



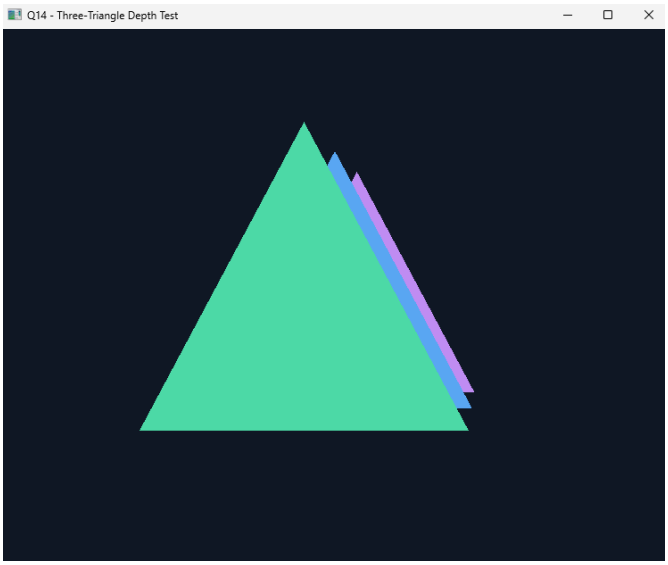
Exercise Q12 — Orbiting Camera via Animated gluLookAt [Medium]

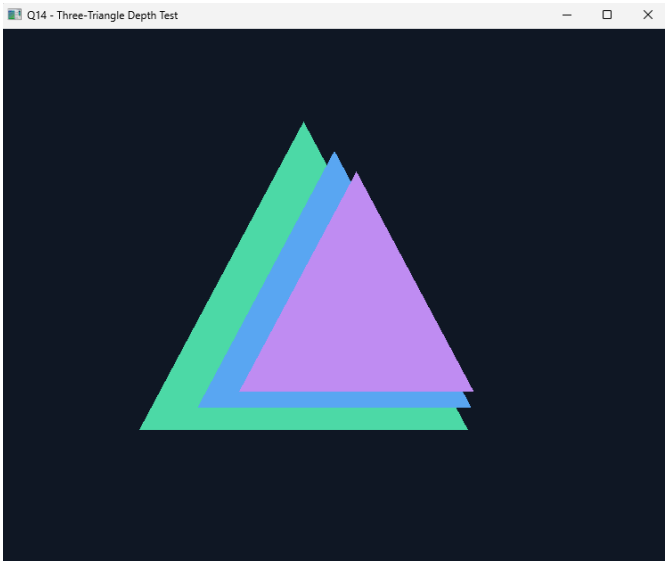


Exercise Q13 — Static Color Quad via glBufferSubData + glArrayElement [Medium]



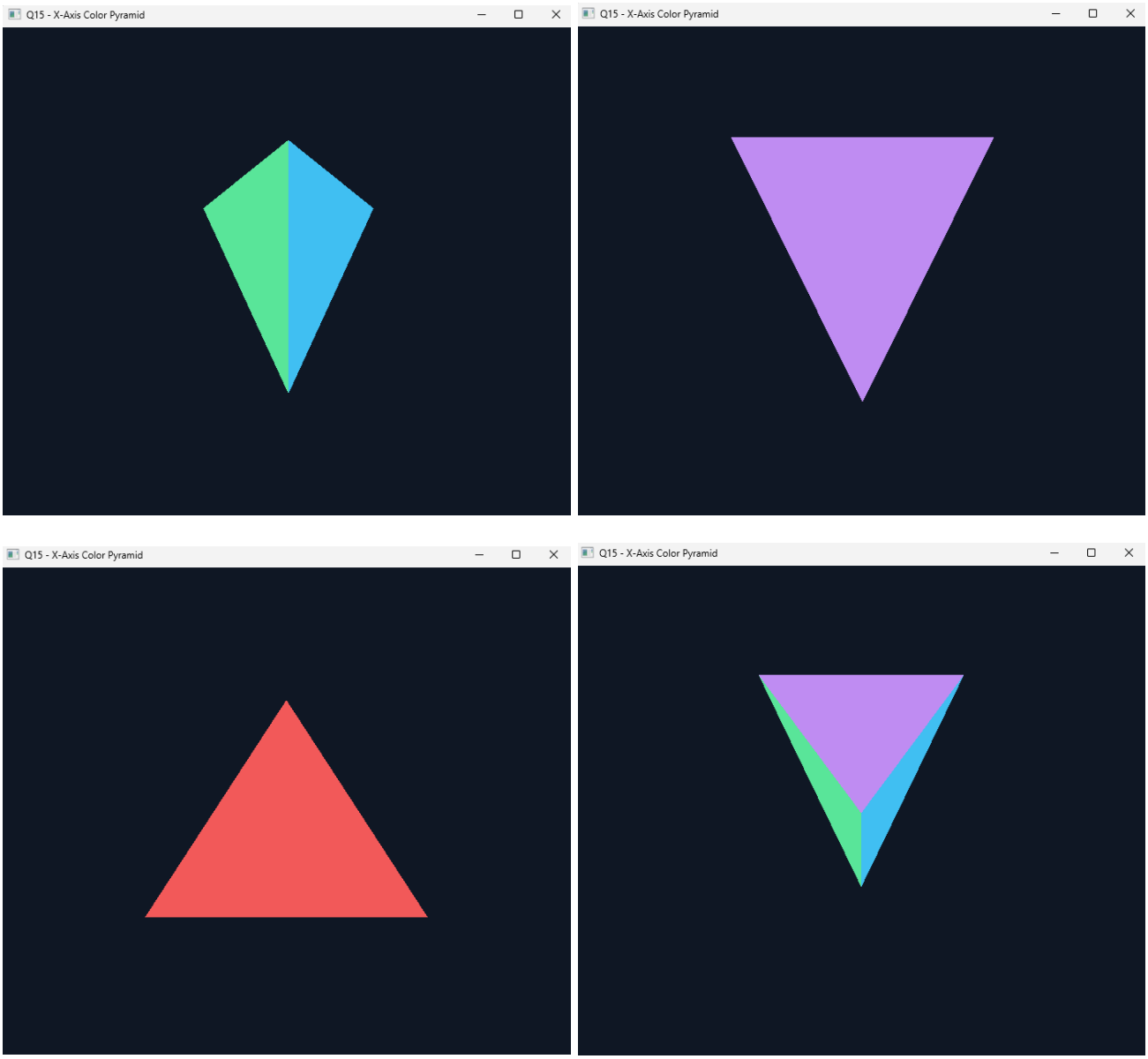
Exercise Q14 — Depth Testing With Three Triangles [Medium]



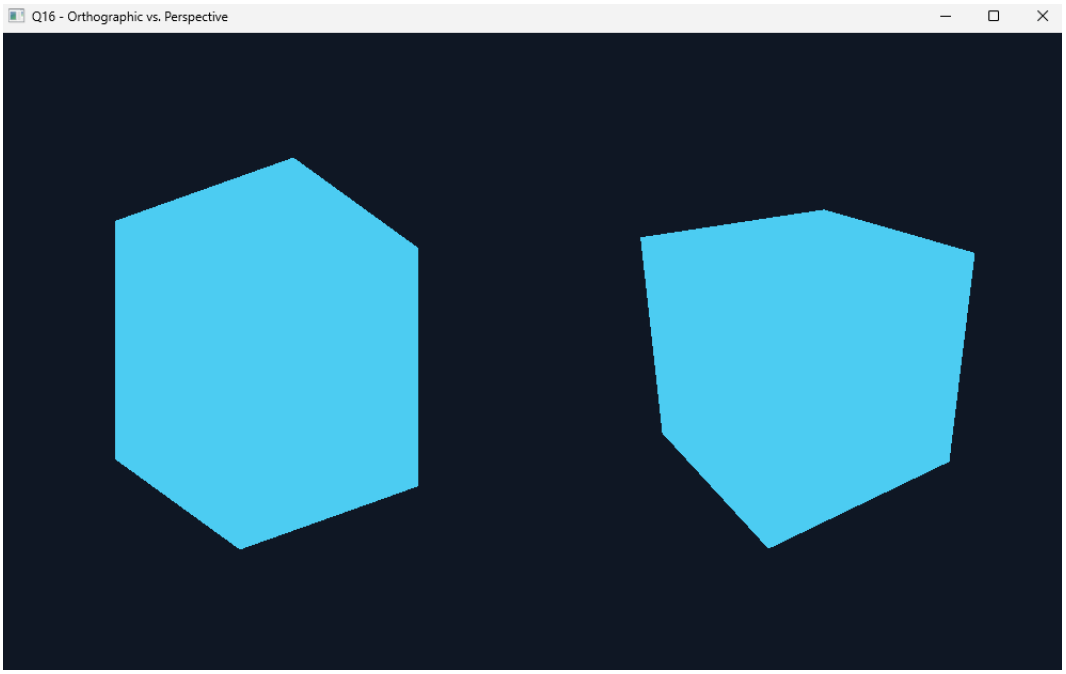


Hard Exercises

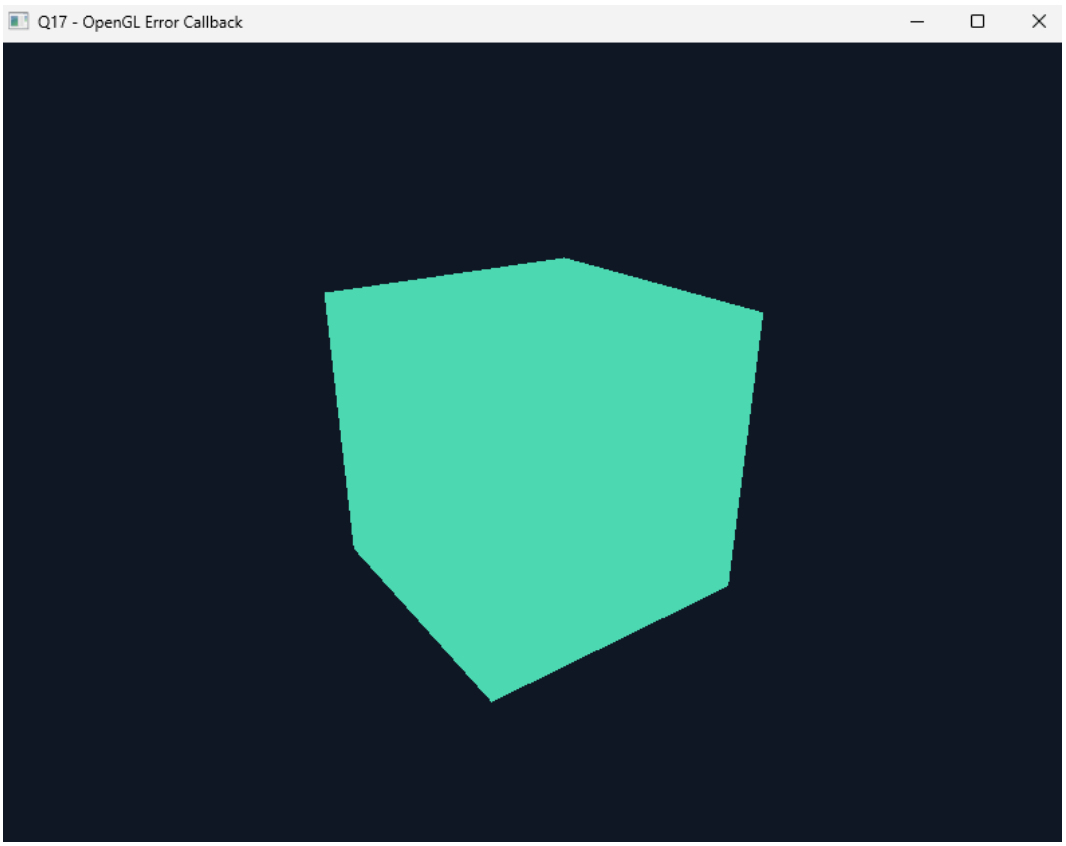
Exercise Q15 — Lecture Pyramid, Rotated on a Different Axis, With Cleanup [Hard]



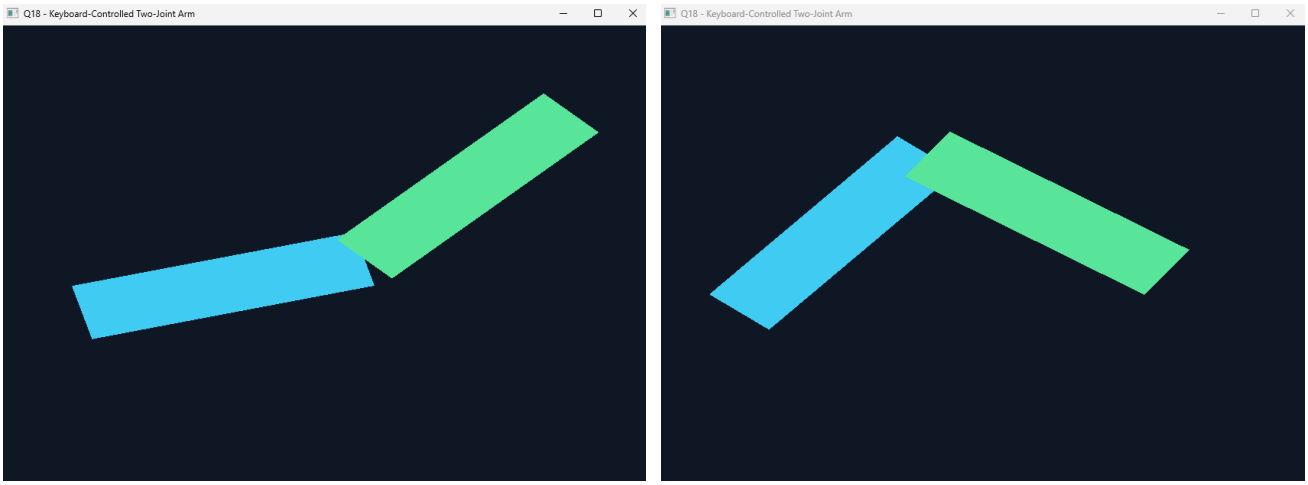
Exercise Q16 — Split Screen: Orthographic vs. Perspective, Side by Side [Hard]



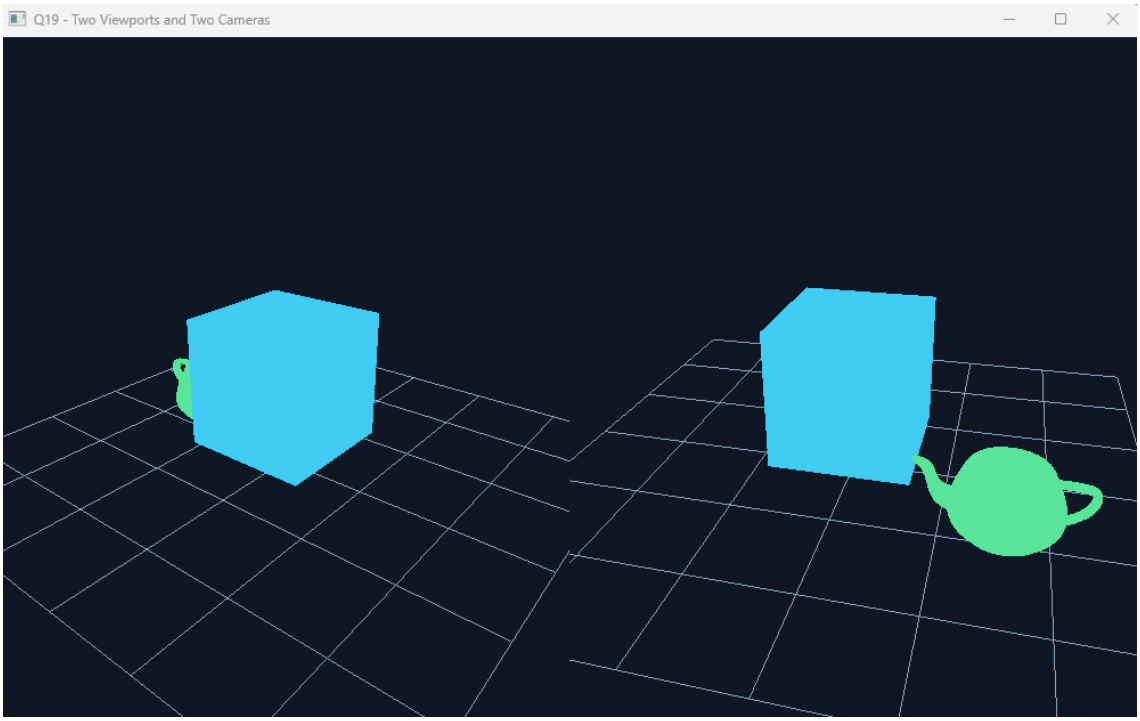
Exercise Q17 — Error Callback With an Intentional Trigger [Hard]



Exercise Q18 — Keyboard-Controlled Two-Joint Arm [Hard]



Exercise Q19 — Two Viewports, Two Cameras, One Animated [Hard]



Exercise Q20 — Capstone: Color VBO Base + Animated Hierarchical Arm [Hard]

